

Rethinking Services in the Quantum Age: The SOQ Paradigm

JOSE GARCIA-ALONSO, ENRIQUE MOGUEL, JAIME ALVARADO-VALIENTE, JAVIER ROMERO-ALVAREZ, ÁLVARO M. APARICIO-MORALES, and JUAN M. MURILLO, Quercus Software Engineering Group, Universidad de Extremadura, Spain

FRANCISCO JAVIER CAVERO, ADRIÁN ROMERO-FLORES, ALFONSO E. MARQUEZ-CHAMORRO, JOSÉ ANTONIO PAREJO, and ANTONIO RUIZ-CORTÉS, I3US Institute, SCORE Lab, Universidad de Sevilla, Spain

GIUSEPPE BISICCHIA, ALESSANDRO BOCCI, and ANTONIO BROGI, University of Pisa, Italy

Abstract. Quantum computing is rapidly progressing from theoretical promise to practical implementation, offering significant computational advantages for tasks in optimization, simulation, cryptography, and machine learning. However, its integration into real-world software systems remains constrained by hardware fragility, platform heterogeneity, and the absence of robust software engineering practices. This paper introduces Service-Oriented Quantum (SOQ), a novel paradigm that reimagines quantum software systems through the lens of classical service-oriented computing. Unlike prior approaches such as Quantum Service-Oriented Computing (QSOC), which treat quantum capabilities as auxiliary components within classical systems, SOQ positions quantum services as autonomous, composable, and interoperable entities. We define the foundational principles of SOQ, propose a layered technology stack to support its realization, a reference architecture for defining quantum services based on the good implementation of a classical service, and identify the key research and engineering challenges that must be addressed, including interoperability, hybridity, pricing models, service abstractions, and workforce development. This approach is of vital importance for the advancement of quantum technology because it enables the scalable, modular, and interoperable integration of quantum computing into real-world software systems independently and without relying on a dedicated classical environment to manage quantum processing.

CCS Concepts: • **Software and its engineering**; • **Theory of computation Models of computation**;

Additional Key Words and Phrases: Quantum Computing, Quantum Software Engineering, Service-Oriented Quantum

Authors' Contact Information: Jose Garcia-Alonso, jgaralo@unex.es; Enrique Moguel, enrique@unex.es; Jaime Alvarado-Valiente, jaimeav@unex.es; Javier Romero-Alvarez, jromero@unex.es; Álvaro M. Aparicio-Morales, amapamor@unex.es; Juan M. Murillo, juanmamu@unex.es, Quercus Software Engineering Group, Universidad de Extremadura, Cáceres, Spain; Francisco Javier Caverro, fcavero@us.es; Adrián Romero-Flores, aromero17@us.es; Alfonso E. Marquez-Chamorro, amarquez6@us.es; José Antonio Parejo, japarejo@us.es; Antonio Ruiz-Cortés, aruiz@us.es, I3US Institute, SCORE Lab, Universidad de Sevilla, Sevilla, Spain; Giuseppe Bisicchia, giuseppe.bisicchia@phd.unipi.it; Alessandro Bocci, alessandro.bocci@unipi.it; Antonio Brogi, antonio.brogi@unipi.it, University of Pisa, Pisa, Italy.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/4-ART

<https://doi.org/10.1145/3805807>

1 Introduction

Quantum computing, based on the foundational contributions of physicists such as Max Planck [46] and Niels Bohr [67], has catalyzed groundbreaking innovation across multiple scientific domains by harnessing the fundamental principles of quantum mechanics [95]. These principles, such as superposition and entanglement, enable quantum computers to process information in ways that differ radically from classical computational models. As a result, quantum algorithms can achieve exponential speedups for certain classes of problems that are intractable for classical machines [21], leading to the redefinition of computational complexity classes such as Bounded-Error Quantum Polynomial Time (BQP) [1]. These emerging capabilities are already beginning to reshape practical fields, including finance [62], pharmacogenetics [75], combinatorial optimization [43], and Artificial Intelligence [88].

Despite the potential, quantum computing remains in its early stages, particularly with the current generation of Noisy Intermediate-Scale Quantum (NISQ) devices [97]. These systems are error-prone, offer limited qubit counts, and require hybrid execution models where quantum and classical computing are tightly linked [52]. To access these quantum capabilities, researchers and developers primarily use cloud platforms offered by companies such as IBM, Google, Amazon, and Microsoft [97]. These platforms enable experimentation and application development through on-demand access to quantum processors, simulators, and hybrid computing environments, democratizing quantum computing for academia and industry alike [58].

However, to take full advantage of quantum computing in real-world systems, it is necessary to integrate it with existing software architectures and paradigms. Service-Oriented Computing (SOC), which has long been a foundation for building scalable, modular, and interoperable software systems, provides a promising path for this integration [56]. The first efforts in this direction defined Quantum Service-Oriented Computing (QSOC) [48, 56], which adapted SOC principles to expose quantum capabilities as abstract and composable services. This adaptation aimed to abstract the technical complexity of quantum back-ends and integrate them as auxiliary components within classical service architectures [78]. QSOC played a foundational role in fostering interoperability, modularity, and reuse in quantum applications, particularly in hybrid computing environments.

Additionally, quantum computing is now maturing to the point where it no longer needs to be framed as an extension to classical systems. The time has come to treat quantum computing not just as a compatible addition to SOC, but as a first-class citizen within service-oriented ecosystems. To this end, we introduce the concept of **Service-Oriented Quantum (SOQ)**, a paradigm that directly inherits the principles of SOC, such as encapsulation, loose coupling, interoperability, and dynamic composition, and applies them to quantum computing in a native way.

SOQ envisions a world where quantum capabilities are exposed, discovered, composed, and orchestrated just like any other service, without requiring them to be anchored in classical service ecosystems. In contrast to QSOC, which emphasized adapting classical methods to incorporate the quantum paradigm, SOQ proposes a dual-native model where classical and quantum services are peers in the architecture. SOQ services are inherently modular and platform-independent, enabling hybrid applications to dynamically interact with quantum and classical back-ends, optimizing for cost, fidelity, and performance. In short, SOQ inherits and directly applies traditional service-oriented principles, treating quantum computing as a first-class participant in service-based ecosystems.

This work aims to lay the theoretical and architectural foundations of SOQ, analyze its differences with respect to previous models such as QSOC, and identify the key challenges and opportunities it brings. Addresses interoperability across quantum platforms, dynamic pricing, hybrid orchestration, and workforce development, critical pillars for enabling SOQ at scale. From this perspective, SOQ is positioned not only as a transitional concept for current NISQ-era systems but as a durable model that will persist into the Fault-Tolerant Quantum Computing (FTQC) era.

The structure of the paper is as follows. Section 2 introduces the foundational concepts and technological components that define SOQ, including quantum computing, software engineering principles, and the hybrid execution model. Section 3 outlines a set of research and engineering challenges that must be addressed to fully realize SOQ, spanning interoperability, platform independence, pricing, and workforce training. Section 4 presents concrete case studies and real-world scenarios that demonstrate the relevance of these challenges across diverse application domains. Section 5 reviews existing work on quantum service abstraction, and we present the challenges that must be overcome to achieve SOQ. Section 6 presents a discussion summarising the challenges identified, highlights interdisciplinary areas among them, and emphasises the research opportunities offered by the interaction between the connected challenges. Section 7 contrasts SOQ with the earlier paradigm of Quantum Service-Oriented Computing (QSOC), highlighting the conceptual and architectural differences, and presenting a proposed technology stack for SOQ, and a reference architecture for the definition of quantum services based on a good implementation of a classical service. Section 8 concludes the paper with a synthesis of insights and directions for future work.

2 Background

To lay a solid foundation for SOQ, we must revisit and clarify the key concepts that distinguish this approach from traditional QSOC. Rather than viewing quantum services merely as extensions of classical Service-Oriented Computing, SOQ posits quantum services as native, independent, and composable building blocks in service-oriented architectures. This section defines the core principles underpinning SOQ, focusing on quantum computing, quantum software engineering, hybridization, services, platforms, and economic models.

2.1 Quantum Computing

Quantum computing is an emerging computational paradigm based on the principles of quantum mechanics, which allow information to be processed using quantum bits (qubits) that exist in superposition and can become entangled. These characteristics enable quantum computers to perform certain types of calculations exponentially faster than classical computers.

The foundational concepts of quantum computation were established in the 1980s by pioneers such as Richard Feynman, who proposed using quantum systems to simulate physical phenomena that classical computers cannot model efficiently [31], and David Deutsch, who introduced the notion of a universal quantum computer [26]. Quantum algorithms such as Shor's algorithm for integer factorization [86] and Grover's algorithm for unstructured search [40] provided the first concrete demonstrations of quantum speedup.

Quantum computing operates on qubits, which can represent a combination of both 0 and 1 states due to superposition. When multiple qubits become entangled, the state of one qubit becomes dependent on the state of another, even at a distance, a property that is crucial for parallelism and interference in quantum computation [61]. These mechanisms allow quantum computers to evaluate multiple possibilities simultaneously and eliminate incorrect outcomes through quantum interference.

However, practical quantum computing faces significant challenges [58]. Current devices, known as NISQ computers [69], are susceptible to noise, decoherence, and gate errors, which limit the depth and reliability of quantum circuits. As such, quantum software must be designed with an awareness of the hardware's physical limitations and the probabilistic nature of measurement.

2.1.1 Quantum Software Engineering.

Quantum Software Engineering (QSE) is an emerging field at the intersection of software engineering and quantum computing [3, 58], focused on the systematic development, testing, deployment, and maintenance of

software systems that involve quantum components. Unlike classical software, quantum software must account for the non-deterministic, probabilistic, and resource-constrained nature of quantum hardware.

Early mentions of QSE framed it as a grand challenge for the discipline [22], anticipating that the rise of practical quantum computers would necessitate entirely new software engineering practices. More recently, researchers have begun to define the scope and processes of QSE, identifying the need for quantum-specific methods in requirements engineering, architecture, programming languages, verification, and maintenance [68, 97].

One of the main distinguishing features of QSE is the inherent hybridity of most real-world quantum software. Current quantum algorithms, for instance, Variational Quantum Eigensolver (VQE) or Quantum Approximate Optimization Algorithm (QAOA), are executed partially on classical processors that coordinate and optimize quantum workloads. As a result, actually, quantum software is not isolated, it must be tightly integrated with classical components, often requiring new software architectures and orchestration models to coordinate quantum and classical execution.

On the other hand, quantum programs are executed on hardware that returns probabilistic results, and measurements collapse the state of qubits, making it impossible to inspect intermediate quantum states without altering them [47]. This limits the applicability of traditional debugging techniques and motivates the development of new validation methods based on statistical testing, simulations, and formal verification.

Furthermore, the lack of mature development environments, IDEs, and toolchains hinders productivity [97]. While frameworks such as Qiskit, Cirq, and Q# provide quantum programming capabilities, they lack many of the lifecycle management tools expected in classical software development [91]. The need for versioning, Continuous Integration and Continuous Deployment (CI/CD) pipelines, modular design practices, and service-based reuse in quantum development is widely acknowledged by the QSE community.

2.1.2 Hybrid Classical-Quantum Systems.

In the current landscape of quantum computing, hybrid classical-quantum systems represent the dominant execution model [56, 90]. Due to the limited capabilities of present-day NISQ devices, characterized by short coherence times, gate errors, and restricted qubit counts, quantum computers are not yet capable of solving most problems independently. Instead, they operate in tandem with classical processors that manage the control flow, data preprocessing, optimization loops, and result interpretation.

A hybrid classical-quantum system typically follows a workflow partitioning model, where specific computational tasks are assigned to quantum processors, such as matrix exponentiation, state preparation, or sampling, while the broader orchestration, including control structures and decision-making logic, remains in the classical domain [11]. This approach enables developers to benefit from quantum acceleration in critical subroutines while retaining the reliability and scalability of classical computing for the rest of the application.

Well-known examples of this pattern include Variational Quantum Algorithms (VQAs) such as the VQE and the QAOA. These algorithms execute parameterized quantum circuits whose outputs are evaluated by a classical optimizer that adjusts the quantum parameters iteratively [55]. This feedback loop exemplifies the tight interdependence between classical and quantum components and illustrates the need for hybrid-aware software architectures.

From a software engineering perspective, the hybrid model introduces several key challenges. First, it is essential to preserve modular separation of concerns, even when classical control logic and quantum operations are tightly interdependent. Second, orchestration layers must accommodate heterogeneous execution paradigms, managing synchronous classical function calls alongside asynchronous quantum job submissions, while ensuring robust scheduling, retry, and failure-handling mechanisms. Finally, data interoperability becomes critical, particularly in the transformation of classical data into quantum states, such as through amplitude or angle encoding, and

the decoding of quantum measurement outputs into usable classical formats for downstream processing and decision-making.

2.2 Service-Oriented Computing

This section introduces the foundational concepts underlying SOC, a paradigm that has played a central role in the development of scalable, modular, and interoperable software systems. The core principles of SOC, such as loose coupling, service abstraction, and composability, are presented as key enablers for constructing complex applications from reusable and discoverable components. These principles are then examined in relation to quantum computing, highlighting the motivation for adopting service-oriented abstractions in quantum software architectures. The subsections that follow provide a structured overview of classical services, quantum services, hybrid classical-quantum platforms, and the relevance of pricing and non-functional properties, establishing the foundation for the subsequent definition and formulation of SOQ as a next-generation architecture model.

2.2.1 Classical Service.

In the field of Software Engineering, a service is a self-contained, platform-agnostic computational unit that exposes a set of functionalities through well-defined interfaces [93]. This concept lies at the heart of SOC, which promotes the decomposition of software systems into modular, reusable, and loosely coupled services that can be dynamically composed and orchestrated [56]. Services are typically designed to be stateless, autonomous, and discoverable, enabling systems to scale, evolve, and integrate flexibly across heterogeneous platforms.

In modern architectures, particularly in cloud-native and microservice-based systems, services may encapsulate business logic, data access, external APIs, or computational models. They are published through service registries, invoked through standard protocols (e.g., REST, gRPC, SOAP), and governed by Service-Level Agreements (SLAs) that define guarantees regarding performance, availability, and cost [33].

From an engineering perspective, services provide:

- Encapsulation. Implementation details are hidden behind standardized interfaces.
- Interoperability. Services can interact across platforms and languages using agreed-upon protocols and data formats.
- Composability. Services can be orchestrated into higher-level workflows or composite applications.
- Reusability. The same service can be used across different domains or projects with minimal adaptation.

2.2.2 Quantum Service.

A quantum service is a modular, remotely accessible unit of functionality that encapsulates quantum capabilities and exposes them through standardized interfaces. Just as traditional services in SOC abstract complex business or computational logic, quantum services abstract quantum computation, enabling users to interact with quantum systems without needing to manage the underlying quantum hardware, circuit representations, or error correction mechanisms.

Quantum services are central to the QSOC paradigm. They allow quantum operations, such as circuit execution, quantum simulations, entanglement generation, or quantum-enhanced optimization, to be integrated into distributed applications as invocable services, decoupled from specific hardware back-ends or programming languages. This abstraction is essential for building interoperable, platform-independent, and reusable quantum software components.

A typical quantum service has the following characteristics:

- Standardized interface (API). The service provides a contract that defines accepted inputs (e.g., circuits, parameters, quantum jobs) and expected outputs (e.g., measurement results, fidelities, state vectors). These

interfaces are commonly implemented using RESTful APIs, OpenAPI specifications, or SDKs in languages like Python or Q# [76, 79].

- **Abstracted execution model.** The service may execute on real quantum hardware (QPUs), simulators, or emulators. Users do not need to know the physical platform used unless explicitly required. The abstraction of the execution model is not complete, and services often need to expose those details, whereas in SOQ, this could be avoided by enhancing the seamless integration between quantum software and its invocation [79].
- **Metadata and constraints.** The service typically includes metadata such as the qubit count, noise level, execution time, number of shots, or queue status. These non-functional properties are essential for orchestrating quantum services within larger workflows.
- **Security and isolation,** like any remote service [96], quantum services must support authentication, authorization, and isolation of workloads. These concerns become critical when multiple users or applications share access to limited QPU resources.

A prominent example of real-world quantum service is Amazon Braket, which allows users to define quantum tasks (several shots of a circuit to be executed in a simulator or quantum computer) in a provider-neutral format and run them on hardware from different vendors. Similarly, IBM Quantum Services exposes quantum circuits through Qiskit APIs, enabling cloud-based quantum computation and integration with classical pipelines. These platforms follow the service model by abstracting back-end complexity and offering programmable quantum capabilities through accessible interfaces [9].

From a Software Engineering viewpoint, quantum services introduce both new requirements and unique challenges [10, 50, 58]. Input and output abstractions must accommodate quantum-specific constructs such as superposition and entanglement, while maintaining compatibility with classical data formats to ensure seamless integration. This need is justified by the fact that quantum functionality is often expressed through quantum oracles that serve as input-dependent quantum transformations. Recent work of the authors on the automatic generation of efficient oracles, for example, the transformation of classical relations such as the “less-than” operator into quantum oracles, demonstrates how classical data can be systematically incorporated into quantum-compatible abstractions [83].

Additionally, error mitigation strategies may need to be embedded within the service layer itself, enabling users to define fidelity targets or trade-offs as part of service invocation. This requirement arises because modern error mitigation techniques are not purely back-end operations; they demand explicit parameterization at the software level. Different works [28, 38] show that methods such as zero-noise extrapolation or probabilistic error cancellation must be configured according to circuit structure, noise characteristics, and desired accuracy, which implies that service interfaces must allow users to specify fidelity thresholds, mitigation modes, and acceptable cost–accuracy trade-offs.

Furthermore, quantum services must expose cost and resource constraints, such as qubit usage, execution time, and queue availability, through the service interface or SLAs, allowing for informed decision-making and negotiation in distributed quantum applications. Studies of quantum cloud platforms demonstrate that queue volatility, hardware availability, and noise fluctuations significantly affect cost and execution quality [37]. Making these constraints explicit at the API or SLA level enables orchestrators and clients to plan executions, optimize scheduling, and reason about performance under uncertainty, a necessity in hybrid classical-quantum environments.

2.2.3 Quantum Computing Platform.

In the context of QSOC, a platform refers to the underlying technological environment that enables the execution, composition, and management of quantum services [59]. While in classical service-oriented computing,

a platform may consist of cloud infrastructure, runtime environments, and service orchestration frameworks, quantum platforms add several layers of complexity due to the heterogeneous nature of quantum hardware, the immaturity of tooling, and the physical constraints of quantum processors.

A typical quantum computing platform includes:

- **Quantum hardware (QPUs).** Devices built on various technologies such as superconducting qubits (e.g., IBM, Rigetti), trapped ions (e.g., IonQ, Honeywell), photonic qubits (e.g., Xanadu), or neutral atoms. Each type offers distinct trade-offs in terms of coherence time, gate fidelity, connectivity, and qubit scalability [49].
- **Execution back-ends.** In addition to real QPUs, platforms often offer high-fidelity simulators or emulators, which are essential for testing and benchmarking quantum algorithms under ideal or noisy conditions. These back-ends can typically be invoked through the same API interfaces as real devices, allowing seamless migration between testing and production environments.
- **Software stacks and SDKs.** Platforms provide development environments (e.g., Qiskit, Cirq, Q#) that include tools for circuit design, compilation, transpilation, optimization, and visualization. These tools are essential for preparing circuits in forms compatible with target hardware constraints (e.g., limited gate sets, qubit topology) [30, 85].
- **Middleware and orchestration.** This includes job schedulers, resource managers, and queueing systems that manage user submissions, ensure fairness, and optimize execution across users and workloads. In multi-tenant cloud environments, these middleware layers enforce quality of service (QoS) policies and enable metering for cost-based access models [30].
- **Monitoring and metering.** Platforms offer dashboards and APIs for tracking usage metrics, queue positions, job outcomes, error rates, and hardware availability. These features support both performance engineering and cost management in production environments.

2.2.4 Pricing Models.

In classical SOC, pricing models are a well-established mechanism that governs the economic interaction between service providers and consumers [33, 35]. These models are typically tied to cloud-based infrastructures, where computational resources, such as CPU time, memory, storage, and network bandwidth, are offered as services on a pay-per-use basis. Common pricing strategies include subscription models, tiered pricing, on-demand billing, and reserved capacity, all of which aim to provide flexibility and predictability for users while optimizing infrastructure utilization for providers.

The design of pricing models in classical computing takes into account several factors, such as:

- **Resource consumption:** execution time, memory, I/O operations, bandwidth, among others.
- **Quality of service (QoS):** availability, latency, and fault tolerance.
- **Service priority:** faster or guaranteed execution for premium users.
- **Scalability:** ability to provision and de-provision resources dynamically based on demand.

This pricing logic is deeply embedded in service orchestration and cloud-native architectures, where it influences deployment strategies or load balancing [8]. Beyond infrastructure-level resources, SOC also encompasses higher-level services, particularly Software as a Service (SaaS), where applications are delivered either through user interfaces (e.g., web front-ends) or programmatic access (APIs). Pricing in this context is often determined by features, plans, and add-ons [35], and operational limits such as request volume, and number of active users [33]. These models extend the principles of flexibility and predictability beyond infrastructure, shaping the economic dimension of application-level services [32].

In the emerging field of QSOC, pricing introduces a new layer of complexity. Quantum computing resources, unlike classical infrastructure, are scarce, expensive, and physically constrained. Quantum processors require specialized environments (e.g., cryogenics), have limited qubit counts, and are prone to errors and decoherence, making them significantly more costly to operate and maintain. As a result, quantum services made available through QSOC must adopt customized pricing schemes that reflect these unique operational realities [80].

Current quantum providers (e.g., IBM, Amazon Braket, IonQ) typically charge based on:

- Number of shots (i.e., repeated circuit executions) and/or number of tasks.
- Qubit usage and circuit depth.
- Number of 1- and 2-qubit gate operations.
- Access tier (e.g., free tier, standard, priority queue).
- Back-end type (simulator vs. real quantum hardware).

In QSOC, where quantum functionalities are exposed as services within classical architectures, pricing must also address the integration of quantum and classical components. This may include billing for hybrid workflows that span multiple execution environments or combining quantum processing with classical preprocessing, orchestration, and result handling. Furthermore, given the non-deterministic nature of quantum outputs and the need for repeated executions, pricing may also incorporate probabilistic guarantees or statistical convergence criteria.

Ultimately, in both classical SOC and QSOC, pricing is not just an economic mechanism but also a driver of resource optimization, service selection, and execution planning. As quantum services become more prevalent within service-oriented ecosystems, pricing strategies will need to evolve to balance fair access, performance guarantees, and operational cost recovery.

3 Interest in Quantum Service-Oriented Computing

Increased efforts have focused on integrating quantum computing with service-oriented principles [78] to address challenges such as interoperability, demand management, and platform independence. This is a direct consequence of the rapid advancement of quantum computing and the growing need for structured solutions. The term SOQ is a novel proposition defined in this work, distinct from the existing QSOC concept. While some early references to QSOC exist from the mid-2010s, due to the 2016 milestone of IBM providing the first quantum computers accessible via the cloud, a more notable mention of QSOC within the field of Quantum Software Engineering appeared in the early 2020s, coinciding with the increasing number of cloud-based quantum computing platforms available.

This has led to a growing interest in this field, with a rise in academic conferences and workshops highlighting the pivotal role of the development of QSOC. The International Conference on Service-Oriented Computing (ICSOC)¹ serves as a prime example of including *Quantum Service Computing* as one of its topics of interest, which reflects the expanding academic activity in the QSOC research area within the service computing community. The convergence of quantum computing and service-oriented architectures has been a key research topic at several other events. This includes the IEEE International Conference on Quantum Software (IEEE QSW)²; the International Conference on Software Engineering (ICSE)³; and workshops such as the International Workshop

¹<https://icsoc2025.hit.edu.cn>

²<https://services.conferences.computer.org/2025/qsw>

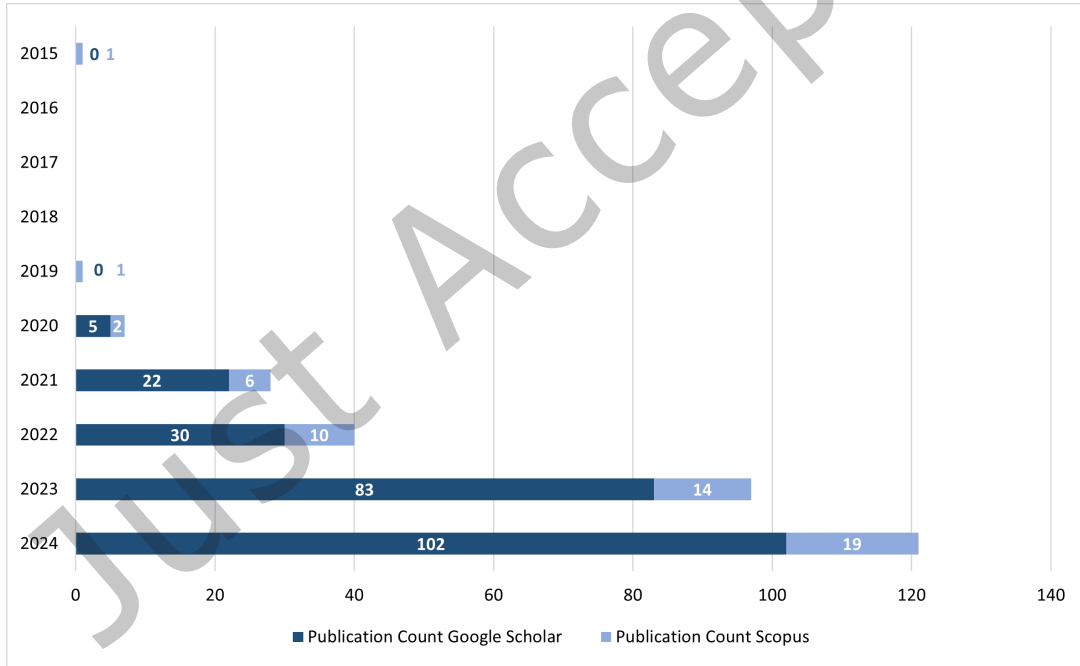
³<https://conf.researchr.org/home/icse-2025>

on Quantum Software Engineering and Technology (Q-SET)⁴, which is part of the IEEE Quantum Week. Furthermore, events such as the Symposium and Summer School On Service-Oriented Computing (SummerSOC)⁵ also underscore the escalating interest in this field, among many others.

Moreover, QSOC-related research topics have been included in some principal journals because of this increasing preoccupation. For instance, the IEEE Transactions on Services Computing and ACM Transactions on Software Engineering and Methodology are two examples where articles exploring the architectural and implementation challenges of quantum services have been published. This trend is also evident in several journals that have dedicated special issues to topics in quantum software, including QSOC, such as the Journal of Systems and Software, the Journal of Information and Software Technology, and the Journal of Quantum Information Processing, among many others.

This growing interest is shown in Fig. 1, which compiles the trend in QSOC publications over the years from two of the leading bibliographic databases, *Scopus* and *Google Scholar*. Specifically, the search was performed on publications that included the terms “*Quantum Service-Oriented Computing*” and “*Quantum Service*” and their possible variants in their content, and which fell within the subject area of computer science or engineering. In Fig. 1, the strong blue bars represent the publication counts from Google Scholar, while the light blue bars correspond to the Scopus search.

Fig. 1. QSOC publications over time.



The trend in QSOC publications shows an initial phase of slow growth from 2015 to 2019, followed by a noticeable increase beginning in 2021 as the potential of quantum computing in service-oriented contexts gained traction. Starting in 2022, this growth became exponential, driven by key factors such as increased funding,

⁴<https://qserv.spilab.es/q-set-2025-home>

⁵<https://www.summersoc.eu>

industry adoption of hybrid classical-quantum systems, the inclusion of QSOC topics in major conferences, and the expanded availability of quantum hardware and platforms. And although 2015 is not yet over, the results obtained suggest that growth and interest in the subject continue.

From this analysis, we would like to highlight different works due to their importance for the scientific community in this field. One of the most cited papers in the field of quantum services is the 2024 paper by Nguyen et al. [60], entitled "*QFaaS: A Serverless Function-as-a-Service Framework for Quantum Computing*," published in Future Generation Computer Systems. The key contribution of this work is the introduction of QFaaS, a comprehensive, vendor-agnostic framework that leverages the serverless model to simplify and standardize the execution of quantum applications. The platform supports a wide range of quantum SDKs, simulators, and cloud providers, and incorporates an adaptive back-end selection policy along with a caching mechanism to mitigate cold start delays, thus addressing practical issues that arise in dynamic quantum workloads.

Another notable contribution in the field is the 2020 paper titled "*TOSCA4QC: Two Modeling Styles for TOSCA to Automate the Deployment and Orchestration of Quantum Applications*", presented at the 24th International Conference on Enterprise Distributed Object Computing (EDOC) [94]. The primary contribution of this work is the proposal of two distinct modeling styles grounded in the TOSCA (Topology and Orchestration Specification for Cloud Applications) standard. These modeling approaches are specifically designed to automate the deployment and orchestration of quantum applications, enabling developers to define and manage quantum workloads in a standardized, declarative manner, paving the way for more scalable and manageable quantum software architectures.

On the other hand, one of the earliest efforts to explore the intersection between quantum computing and service-oriented architectures is the 2015 paper by Jatoth et al. [45], titled "*QoS-Aware Web Service Composition Using Quantum-Inspired Particle Swarm Optimisation*", presented at the 7th KES International Conference on Intelligent Decision Technologies (KES-IDT 2015). In this work, the authors propose a novel approach to solving the QoS-aware web service composition problem using a quantum-inspired particle swarm optimization (QPSO) algorithm. This pioneering contribution laid the groundwork for future research by demonstrating how principles derived from quantum computing could be applied to enhance the performance of service composition mechanisms, particularly in multi-objective optimization contexts.

Finally, a more recent work, published in 2024, was entitled "*A Reference Architecture for Quantum Computing as a Service*", published in the Journal of King Saud University – Computer and Information Sciences. In this work, Ahmad et al. [2] propose a set of design guidelines and recommendations for building quantum-enabled microservices based on the classic-quantum split pattern. The paper's key contributions include an empirically grounded reference architecture and a proof-of-concept implementation that demonstrates how quantum and classical components can be modularized into interoperable microservices. This work offers a practical blueprint for developing hybrid applications and contributes to the broader discourse on architecture patterns for Quantum Computing as a Service (QCaaS).

4 Influence of the SE2030 Roadmap Workshop

This paper is significantly influenced by the discussions and insights outlined during the "SE2030 Roadmap Workshop", a strategic event co-located with ACM SIGSOFT FSE on June 26-27, 2025, in Trondheim, Norway⁶. This gathered leading software engineering researchers to collectively define a vision for the discipline for the coming years. The workshop identified different foundational areas that will reshape software engineering: Agentic AI, AI for SE, SE for and by Humans, Sustainable SE, Quantum Software Engineering, among others. Each of these areas revealed tensions between established practices and the transformative impact of emerging technologies, including quantum computing.

⁶2030 Software Engineering (2025). Retrieved from <https://conf.researchr.org/home/2030-se-2025>

The QSE discussions at the workshop recognized that quantum computing represents not just a technical evolution, but a change of paradigm that questions foundational assumptions of software design, architecture, and lifecycle management. One recurring theme was the necessity to rethink modularity and abstraction in light of quantum-specific constraints such as decoherence, entanglement, and probabilistic computation. While traditional software engineering relies on deterministic execution and well-defined state transitions, quantum software forces us to build systems under uncertainty, incomplete observability, and limited introspection capabilities. The SOQ model responds directly to this shift by proposing a service-oriented architecture that abstracts quantum resources as composable and discoverable services, shielding classical developers from the low-level mechanics of quantum logic gates and error models.

Participants recognized that while hybrid classical-quantum architectures are essential in the current NISQ era, the long-term vision for QSE should not be confined to hybrid systems. In contrast to models that treat quantum systems as auxiliary to classical workflows, SOQ aims to establish an architectural paradigm where quantum services can exist and operate independently, whether in quantum environments or in hybrid ecosystems. This independence is enabled by SOQ's emphasis on loose coupling, platform-agnostic service definitions, and modular orchestration mechanisms. Several researchers at the workshop highlighted the need for robust intermediate layers, such as standardized quantum APIs, middleware, and orchestration frameworks, that not only bridge quantum and classical components when needed but also support fully quantum-native service ecosystems. These abstractions are central to SOQ's identity, ensuring that quantum services remain portable, composable, and interoperable across both heterogeneous back-ends and diverse deployment contexts.

On the other hand, the discussions around AI for SE and Agentic AI revealed clear intersections with SOQ. For example, AI tools, such as Large Language Models (LLMs) and program synthesis engines, can be used to automate the generation of quantum circuits or hybrid service compositions. The opportunity to leverage Generative AI for designing SOQ service interfaces, configuring workflows, and generating test cases for probabilistic services was highlighted. At the same time, the validation of such systems presents novel challenges. As highlighted, the inherently non-deterministic behavior of quantum services requires new frameworks for correctness, trust, and confidence. The SOQ model can serve as a testing ground for advancing such verification models, especially in hybrid service contexts where observability and reproducibility are limited.

Additionally, the proposal for human-centric software engineering is particularly relevant to SOQ, which must navigate the tension between complex quantum principles and the goal of providing accessible, understandable interfaces to developers and domain experts. A topic of interest was the idea of “quantum usability layers” that translate quantum operations into domain-level constructs, such as finance, logistics, or chemistry, facilitating broader adoption. SOQ can offer a pathway for implementing these usability layers as modular services that encapsulate domain-specific quantum capabilities (e.g., optimization, simulation) without requiring users to understand the underlying hardware or algorithms.

On the sustainability front, the energy-intensive nature of quantum computing was flagged as a key concern. While quantum algorithms may reduce the time complexity of certain tasks, the overhead of cooling quantum processors, running redundant shots, and maintaining coherence across qubits can offset these benefits. In this regard, SOQ can act as a governance and optimization layer, managing resource allocation and execution policies based on sustainability metrics (e.g., energy per useful operation, carbon footprint per service call, etc.). This could enable future research on green quantum computing, where orchestration policies account for both performance and ecological impact.

Ultimately, the SE2030 workshop underscored the need for shared frameworks, standards, and open toolchains that allow the SE community to engage meaningfully with quantum computing. The SOQ vision is a tangible response to this demand, it encapsulates quantum capabilities within a familiar and extensible architectural model, enables hybridization through well-understood service composition patterns, and promotes research into new models of validation, security, and sustainability.

In short, all these observations and discussions that emerged during the SE2030 workshop strongly resonated with the authors' current work on this paper, motivating us to explicitly position SOQ as a first-class service-oriented paradigm for quantum computing, going beyond the previous concept of QSOC. This paper incorporates many of the challenges articulated in the workshop, which are detailed and expanded upon in the following sections of this manuscript.

5 Challenges in Service-Oriented Quantum

The SOQ paradigm leverages the principles of SOC to the quantum domain, ensuring modularity, scalability, and interoperability in hybrid classical-quantum environments. Unlike previous approaches that treated quantum services as adaptations within SOC, SOQ positions quantum computing as a first-class participant in service-oriented ecosystems. This shift enables quantum functionalities to be exposed as independent, loosely coupled services that can be composed, orchestrated, and integrated seamlessly with classical IT infrastructures.

However, several challenges must be addressed to fully realize the benefits of SOQ. These include the need for standardized quantum service interfaces, efficient classical-quantum communication protocols, and adaptive orchestration mechanisms that can handle the dynamic nature of quantum computations. Furthermore, given the limitations of NISQ devices, SOQ must accommodate error-prone quantum services while ensuring fault tolerance and reliability in hybrid workflows. Addressing these challenges will be essential to developing a robust and efficient Quantum-as-a-Service (QaaS) ecosystem, ultimately enabling broader adoption of quantum technologies across various industries.

5.1 Relevant works that address the challenges

The diverse research efforts addressing different challenges and aspects, from architecture design to practical implementations, reflected the growing interest in this area, as seen before. This section reviews some of the most relevant contributions in this field that seek to solve the problems and challenges detected. Fig. 2 shows a correlation between SOQ challenges, sub-challenges, the use cases raised, and the most relevant research works. As far as we know, there are still challenges that have not yet been addressed, which reaffirms the crucial need for future work and studies in this area.

Some of the earliest works exploring the intersection of SOC and quantum computing are by Karoline et al. [94], where the authors propose several modelling styles based on the TOSCA standard. They discuss the challenges of orchestrating and deploying quantum and classical services, emphasizing the need for interoperability mechanisms. Moreover, in Kumara et al. [48], the authors present a model-driven methodology based on QSOC that allows developers to build hybrid enterprise applications collaboratively.

In studies focused on the coexistence of classical and quantum services, the work of Ali and Yue [4] stands out, where they emphasize the need for a quantum-oriented paradigm to address challenges and highlight key issues such as interoperability, abstraction mechanisms, and platform independence. Moreover, in Romero et al. [74], a scheduler is proposed to execute quantum circuits on cloud service providers. Similarly, Alvarado et al. [5, 6] present a guide to converting quantum circuits into web services. Furthermore, the study by Nguyen et al. [60] introduces the concept of QFaaS, leveraging the serverless model for the execution of quantum circuits.

Architectural patterns have also been explored from the perspective of workflow and platform design. Delgado et al. [25] discuss architectural constraints for hybrid workflows and propose design primitives for mapping algorithmic stages across heterogeneous back-ends. The work of Saurabh et al. [84] provides an architectural view of quantum middleware and describes abstractions necessary for portable circuit execution, while Ahmad et al. [2] outline a reference architecture for QCaaS integrated within cloud platforms.

Orchestration is a key challenge in SOQ, as quantum workflows require sophisticated middleware solutions to manage heterogeneous quantum back-ends. Some works address this issue by proposing orchestration frameworks for quantum applications, which enable workflow automation and efficient resource allocation [51, 92].

An alternative approach to managing hybrid classical-quantum systems, while also improving interoperability, focuses on distributing the quantum workload on a *shot-by-shot* basis, rather than sending an entire quantum job to a single QPU. This idea is explored in the work by Bisicchia *et al.* [15, 18] in which the authors introduce a *Quantum Broker* that automatically splits the requested number of shots among multiple QPUs according to user-customizable policies. Allocating subsets of the total shots to different QPUs improves both flexibility and resilience. If one QPU experiences downtime or failure, its share of shots can be reallocated to functioning machines, and only a fraction of the overall results are affected.

Similarly, DevOps methodologies have been adapted to quantum computing, incorporating automated deployment [12], containerization [89], and delivery practices to ensure software reliability in hybrid classical-quantum environments [7, 77]. These studies emphasize the need for automated testing and validation of quantum circuits to mitigate the instability of quantum hardware.

Regarding quantum service deployment, recent research has also explored the modularization of hybrid information systems to ensure platform compatibility [66]. Other studies have investigated testing implications [57] and the quality of deploying quantum services [27] in cloud environments, proposing best practices to test quantum circuits.

Collectively, these works establish the interest, the need to integrate quantum and classical systems, and the technical feasibility, but above all, more efforts are needed in this area. Therefore, based on the current work of the community and the authors' experience in this field over recent years, several challenges have been identified. The main ones are as follows.

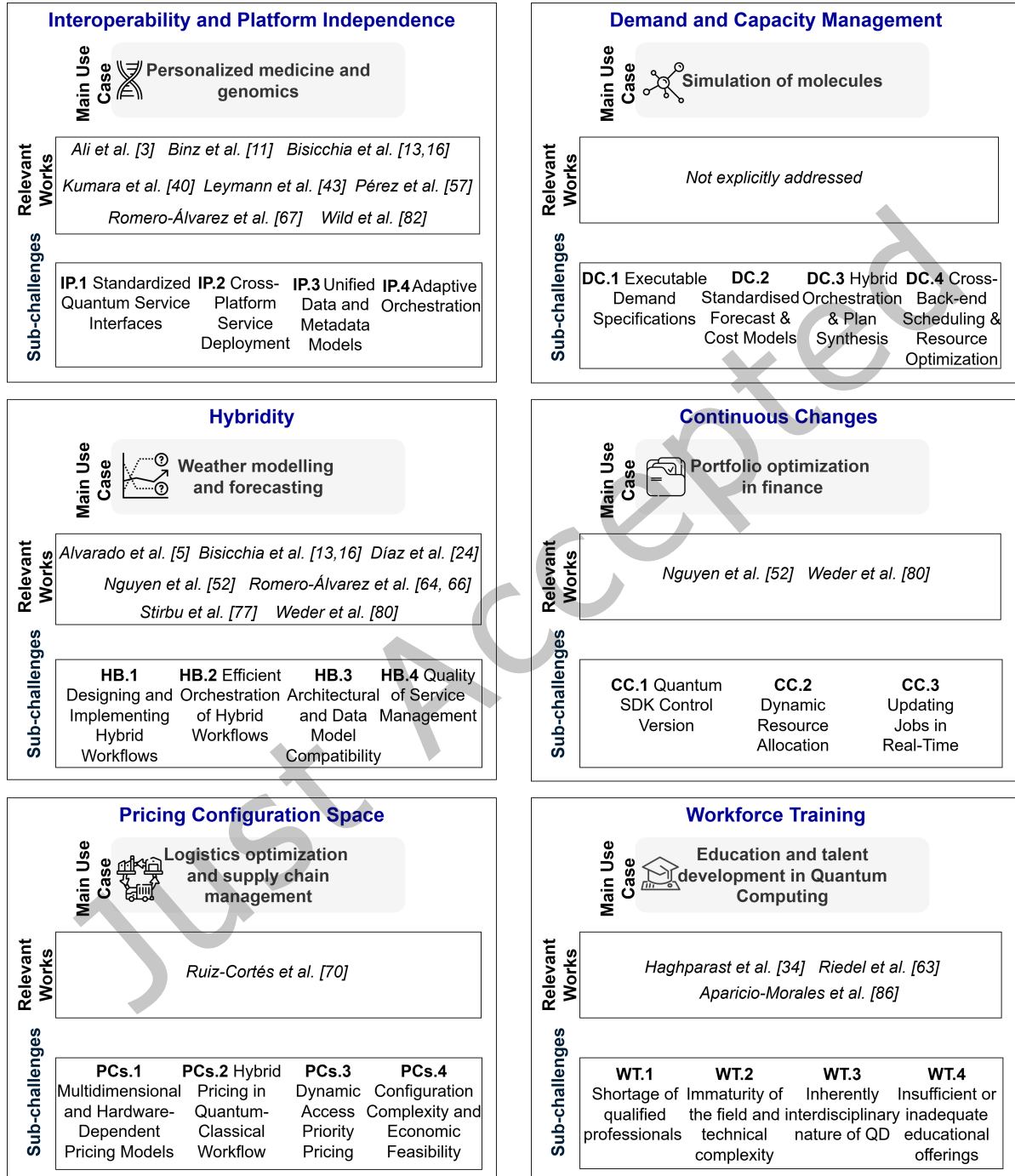
5.2 Interoperability and Platform Independence

Interoperability and platform independence stand as foundational pillars for realizing the Service-Oriented Quantum paradigm. Without robust mechanisms to integrate heterogeneous quantum and classical resources, the promise of scalable, reusable, and maintainable quantum-enabled services remains unattainable. Yet the present quantum computing landscape is characterized by severe fragmentation [97], the risk of vendor lock-in, and rapidly evolving software stacks, factors that continue to impede the emergence of open, flexible, and accessible quantum infrastructures.

The ecosystem is fragmented mainly due to a tendency by quantum providers to develop their own QPUs, software stacks, SDKs, and languages [41, 85]. This diversity has generated a proliferation of proprietary interfaces and non-standardized circuit representations, resulting in programming models and toolchains that are usually challenging to integrate across platforms. The absence of standardized abstractions manifests in several problematic ways [19]. Algorithms, workflows, or services created for one provider are rarely portable to others without substantial manual re-engineering, which undermines the vision of reusable, cross-platform quantum software. Users are often confined within a single vendor's ecosystem, restricting their capacity to select the most suitable hardware for a given problem and, in turn, stifling both competition and innovation. Moreover, the research community suffers from duplicated efforts and missed opportunities for synergy, as solutions developed in one stack are typically not composable with those from another, fragmenting progress and limiting large-scale adoption.

Addressing these challenges, the community has started to pursue standard intermediate representations and interoperability layers. Solutions such as OpenQASM [24], the Quantum Intermediate Representation (QIR) [53], and toolkits like *(t|ket)* [87] and XACC [54] represent promising steps toward enabling quantum circuit

Fig. 2. Challenges, possible use cases, and relevant works addressing the challenges.



portability across back-ends. Provider-neutral SDKs, such as qBraid⁷ and QuantumExecutor [14], attempt to abstract away some of the provider-specific complexities. Nevertheless, progress is uneven. Even when nominal compatibility exists, differences in supported gate sets, qubit connectivity, and error models may introduce semantic mismatches, which can degrade performance or even yield incorrect results. Furthermore, because the execution of quantum services remains closely tied to the physical characteristics of specific QPUs (such as their topology, coherence times, and susceptibility to noise) true platform independence remains an elusive goal.

A promising strategy to address both interoperability and platform independence is the introduction of meta-platforms or “Virtual Quantum Providers” (VQPs) [19]. These act as intermediaries that abstract over the diversity of providers, exposing unified interfaces and automating provider selection, resource management, and job routing. Pioneering examples include qBraid, Classiq⁸, PlanQK [29], and orchestration frameworks that support multi-stack software pipelines.

These VQPs provide several key advantages:

- **Unified API:** Developers can target a single programming interface, while the VQP handles translation, compilation, and optimization for multiple hardware targets.
- **Automated provider selection:** Algorithms for cost, performance, or availability can dynamically select the best QPU for each task, maximizing resource utilization and minimizing queue times [13, 17, 18].
- **Reduced vendor lock-in:** Decoupling application logic from specific providers reduces migration costs and future-proofs software investments.
- **Composable pipelines:** Multi-stack workflows become feasible, allowing compilation, simulation, and execution stages to leverage the unique strengths of various SDKs and providers in a seamless fashion [15].

Yet, challenges remain. VQPs must cope with evolving provider APIs, changing hardware capabilities, and the ongoing lack of robust, community-wide standards. The abstraction gap can mask, but not fully eliminate, semantic mismatches and performance inconsistencies rooted in the physical layer.

Building on these insights, we identify several concrete research challenges to advancing interoperability and platform independence within SOQ architectures (Table 1).

Table 1. Interoperability and Platform Independence Challenges for SOQ.

#	Challenge	Description
IP.1	Standardized Quantum Service Interfaces	Define and formalize universal, provider-agnostic interfaces for quantum services, including inputs/outputs, resource requirements, and quality attributes.
IP.2	Cross-Platform Service Deployment	Develop abstraction and deployment mechanisms enabling seamless execution of quantum services across diverse providers.
IP.3	Unified Data and Metadata Models	Create interoperable data formats for classical-quantum exchange, along with standardized metadata to describe job context, fidelity, and provenance.
IP.4	Adaptive Orchestration	Implement orchestration engines capable of real-time provider selection, resource negotiation, and workflow optimization based on availability and performance demands.

⁷<https://github.com/qBraid>

⁸<https://www.classiq.io>

IP.1 Standardized Quantum Service Interfaces

A fundamental step toward interoperability is the establishment of standardized, universal service definitions-analogous to GraphQL [71], REST or OAS in classical SOC. This involves specifying canonical types for quantum inputs (e.g., circuits, oracles, parameterized gates), outputs (bitstrings, state vectors, error estimates), and service metadata (back-end topology, calibration data). Such standardization would allow developers to compose and reuse services independent of the underlying hardware or SDK, while supporting the automatic negotiation of resource needs and QoS expectations [17].

IP.2 Cross-Platform Service Deployment

Realizing platform independence requires not only standard interfaces, but also deployment mechanisms that enable quantum services to run “anywhere”. Techniques such as intermediate circuit representation, automatic transpilation, and runtime adaptation to target-specific constraints (e.g., gate sets, qubit topology) are needed. Existing compilers like (*t|ket*) and frameworks like ProjectQ provide some of this flexibility, but further work is required to make these solutions transparent, robust, and integrated within the SOQ lifecycle.

IP.3 Unified Data and Metadata Models

A neglected but critical aspect is the standardization of data models for classical-quantum exchange. Measurement results, error rates, and calibration data must be represented in forms that are compatible and easily consumable by classical services, with rich metadata to support traceability, reproducibility, and statistical analysis. Without such standards, hybrid workflows will remain brittle and error-prone.

IP.4 Adaptive Orchestration

Future SOQ platforms may include orchestration engines that monitor hardware status, predict queue times and error rates, and dynamically re-route jobs to the most appropriate provider [18]. This requires integration of hardware monitoring APIs, predictive analytics, and policy-driven resource allocation [17]. The end goal is “liquid” quantum services, adaptively migrating, scaling, and optimizing execution based on a continuously changing environment.

While the push for interoperability and platform independence is widely supported, a well-informed skeptic might argue that too much abstraction risks diluting performance, obfuscating critical hardware-specific optimizations, and creating a “lowest common denominator” effect that undermines practical utility. Furthermore, as the field is rapidly evolving, premature standardization could lock the ecosystem into suboptimal patterns. It may be argued that targeted, use-case-specific pipelines, rather than universal abstractions, will remain necessary for some time, especially in high-stakes applications where performance or reliability is paramount.

Nonetheless, history in classical computing suggests that the long-term benefits of interoperability (such as reduced duplication, improved reproducibility, and faster innovation cycles) outweigh the temporary losses from abstraction. The challenge for SOQ is to balance these trade-offs: enabling portability and composability without sacrificing the ability to leverage unique quantum hardware capabilities.

Interoperability and platform independence are indispensable for the maturation of quantum software engineering and the realization of SOQ. Overcoming the current landscape of fragmentation, vendor lock-in, and brittle software stacks will require sustained efforts in standardization, orchestration, and community collaboration. Virtual Quantum Providers, unified service interfaces, and adaptive orchestration engines represent promising paths forward, but require careful design to avoid masking fundamental performance or reliability gaps. Ultimately, addressing these challenges is not merely a technical necessity, it is an enabler for the practical

impact of quantum computing across scientific, industrial, and societal domains. A unified, interoperable SOQ layer would allow seamless migration, composition, and scaling, accelerating discovery and democratizing access to cutting-edge computational resources.

5.3 Demand and Capacity Management

Typically, quantum computing relies on best-effort runs on a single back-end: circuits are executed multiple times (called *shots*), queued by providers, and in hybrid workflows interleaved with classical computation [61]. Service-Oriented Quantum aims to make resource management and service time explicit, to guide the planning and execution of quantum workloads across heterogeneous providers.

Quantum computing resources remain scarce and expensive. Unlike classical cloud computing, where resource allocation and autoscaling are well understood, quantum capacity faces hardware-specific constraints and rapidly changing operational conditions. In particular, *service times*, *gate fidelities*, and *qubit availability* jointly determine the *effective* throughput of a quantum service [97]. Queue-based execution and heterogeneous pricing introduce cost-latency trade-offs that demand explicit management, while hybrid jobs add classical billing to the mix. Adapting demand-management patterns from microservice architectures to this context calls for new abstractions for quality-aware scheduling, budget control, and classical-quantum co-scheduling across *heterogeneous* devices and providers.

SOQ aims at making these levers explicit and actionable. End-to-end performance is governed by the *service time* budget:

$$T_{\text{svc}} = T_{\text{queue}} + T_{\text{quantum}} + T_{\text{classical}}$$

Where, T_{queue} reflects provider scheduling, concurrency caps, and priority tiers, whereas $T_{\text{classical}}$ covers optimization, data movement, and hybrid pre/post-processing in hybrid loops. Crucially, T_{quantum} must be understood as the combination of several hardware-dependent stages rather than a single undifferentiated value. The total quantum execution time is shaped by the duration of qubit preparation, the speed of gate operations, the overhead introduced by routing due to limited qubit connectivity, the latency of measurement, and the time required to reset qubits before reuse. These stages behave very differently across quantum technologies: superconducting devices can initialize qubits quickly and execute gates within tens of nanoseconds, while trapped-ion machines require much slower, microsecond-scale gates but benefit from all-to-all connectivity that reduces routing overhead. Photonic systems often exhibit negligible gate duration yet incur high measurement latency, whereas neutral-atom platforms introduce timing constraints linked to atomic transport and optical control. The number of shots required to achieve the desired statistical confidence multiplies all these contributions and depends on the noise characteristics and calibration quality of the device. Because these parameters vary so widely across superconducting, trapped-ion, neutral-atom, and photonic processors, the latency and cost of executing the same quantum service can differ by orders of magnitude depending on the selected backend.

This variability makes explicit modeling essential within SOQ. Therefore, without decomposing T_{quantum} developers cannot reliably predict performance, determine when quantum execution is advantageous, or make governance-aware scheduling decisions. Understanding these components also opens concrete avenues for QSE research, ranging from predictive models that estimate execution time under different noise profiles, to methods for navigating fidelity–latency trade-offs, to device-agnostic abstractions for expressing performance requirements, to schedulers capable of reasoning about connectivity, gate duration, and queue times, and ultimately to hybrid orchestrators that can decide when quantum execution is truly cost-effective compared to classical alternatives.

Effective demand and capacity management in SOQ thus becomes essential to optimize resource allocation, ensure efficient classical-quantum task scheduling, and mitigate the high operational cost of quantum processing.

Therefore, effective planning must treat fidelity, service time, and qubit availability as first-class controls: schedule submissions when calibration is favorable, route circuits to qubit pairs with higher two-qubit fidelities

that are currently enabled, and, when feasible, distribute shots across eligible back-ends to reduce wall-clock time and hedge against transient noise. Pricing choices are inseparable from these decisions: large, precision-driven shot portfolios with tight SLAs may justify reservations, and exploratory runs often benefit from on-demand and dynamically priced options, paying only for the shots and tasks actually executed. To make such decisions reproducible and auditable, SOQ services should expose *capability contracts* that an orchestrator enforces at admission and uses to steer compilation, routing, scheduling, and measurement planning. This *capability-based programming* style allows requesters to express priorities over *heterogeneous* resources (e.g., preferring calibrated qubit pairs or bounding queue delays) and to align execution with domain goals.

The current research on capacity management focuses predominantly on QPU selection by optimizing metrics over devices, circuits, and environment (e.g., queue length, cost, availability). Garcia-Alonso et al. [34] select QPUs given architecture and circuit width while letting users prioritize speed or cost. Marie et al. [81, 82] go further by selecting both a circuit implementation and a QPU based on width, depth, SDK compatibility, and input-dependent rules, with extensions for compiler comparison and ML-based pruning/optimization. Other proposals jointly consider estimated output fidelity and queue waiting time in adaptive schedulers [72], integrate quantum within enterprise cloud selection using static attributes such as qubit count and queue length [39], or predict the best combination of QPUs, compilers, and compilation parameters to maximize execution fidelity [70]. Despite these advances, most approaches still choose a *single* best QPU (or QPU–circuit pair) and execute all measurement shots there. By contrast, *shot-wise distribution* [16, 18] exploits the statistical independence of shots to distribute them across multiple *heterogeneous* QPUs, enabling parallelism, resilience to device drift, and adaptive load balancing, an approach that aligns naturally with SOQ’s governance of service time, fidelity, and price.

Building on the above discussion, we identify four concrete research challenges that advance demand and capacity management within SOQ (Table 2). These challenges translate the execution model (shots, queues, hybrid loops), service-time budgeting, hardware quality (fidelity and qubit availability), and pricing choices into actionable design problems for governed, repeatable operation across *heterogeneous* devices and providers.

DC.1 Executable Demand Specifications

Provide a capability-based, machine-checkable specification through which users declare what the service must achieve and under which bounds. Typical fields include service time, SLA targets, price ceilings, maximum shots, preferred pricing mode (on-demand or reserved), fidelity targets, and eligible devices/providers. Specifications should support composition (pipelines of circuits and hybrid loops), be statically validated before admission, and remain auditable during execution so that decisions (e.g., rerouting or early stopping) can be traced back to declared constraints.

DC.2 Standardised Forecast & Cost Models

Define provider interfaces that expose pre-execution forecasts for queue time, quantum run time (per circuit and per shot), classical overhead for hybrid steps, expected monetary cost (per-shot/per-task and reservation rates), and expected computation fidelity. These interfaces should return point estimates with confidence intervals, specify the forecasting horizon and assumptions (e.g., current queue state), and support what-if queries (e.g., “50k shots across two back-ends”). Standardization enables planners to compare options across providers and to produce consistent, auditable quotes prior to submission.

DC.3 Hybrid Orchestration & Plan Synthesis

Develop orchestration services that translate user specifications and provider forecasts into executable plans spanning quantum and classical resources. Plans should decide when to submit (or reserve), which back-ends to target, how to partition and order shot batches, how to align classical optimization

Table 2. Demand and Capacity Management Challenges for SOQ.

#	Challenge	Description
DC.1	Executable Demand Specifications	User-facing, provider-agnostic ways to express execution priorities and constraints (e.g., latency targets, spend ceilings, fidelity targets, eligible devices, maximum shots, reservation requirements) that an orchestrator can validate, audit, and enforce.
DC.2	Standardized Forecast & Cost Models	Cross-provider interfaces exposing forecastable service attributes, queue delay, run time, price, and expected fidelity, with uncertainty bands and what-if queries, enabling apples-to-apples planning and pre-execution quotes.
DC.3	Hybrid Orchestration & Plan Synthesis	Orchestrators that derive end-to-end execution plans consistent with user specifications: selecting devices and pricing modes, sizing and ordering shot batches, aligning classical steps, and preparing distribution strategies across <i>heterogeneous</i> back-ends.
DC.4	Cross-Backend Scheduling & Resource Optimization	Scheduling policies and algorithms that allocate and re-balance workload (including shot-wise distribution) to optimize cost/latency under constraints, while coping with concurrency caps, calibration changes, and multi-tenant fairness.

steps with expected device availability, and whether to employ heterogeneous distribution. They must include rollback and adaptation rules (e.g., switch device if availability changes) and produce artifacts for governance: a rationale, predicted T_{svc} and spend, and checkpoints for mid-course corrections.

DC.4 Cross-Back-end Scheduling & Resource Optimization

Design schedulers that implement the plan under real-time conditions, allocating jobs and shots across multiple eligible back-ends to minimize cost and latency while respecting constraints from DC.1. Key capabilities include queue-aware admission control, fidelity-aware routing and compilation choices, adaptive shot allocation with early stopping, and on-the-fly rebalancing when queue states, calibration outcomes, or availability change. The scheduler should support multi-tenant fairness and provide utilization metrics that feed back into forecasting and future plans.

Taken together, these elements point to SOQ as a practical pathway from best-effort runs to predictable, cost-aware quantum services. It achieves this by making critical metrics, such as service time, gate fidelity, and qubit availability, explicit and by capturing user intent through executable demand specifications. In addition, SOQ exposes provider forecasts and cost models, integrates hybrid orchestration, and enables cross-back-end scheduling, including shot-wise distribution. Through this combination, SOQ aligns pricing and latency with accuracy objectives across heterogeneous providers. The outcome is a repeatable and auditable execution process that mitigates scarcity and noise, improves resource utilization, and allows domain teams to meet service-level agreements without compromising scientific or industrial objectives.

5.4 Hybridity

Hybrid classical-quantum workflows require efficient coordination between systems. Many quantum algorithms rely on classical pre-processing and post-processing steps, creating potential bottlenecks in service orchestration.

Recent research efforts have started to address some of these challenges by proposing early architectures and runtime environments for hybrid execution. Examples include AWS Hybrid Jobs and IBM Quantum Runtime, which provide frameworks for combining quantum and classical computations in a coordinated way, and the Qubernetes platform [89], which explores cloud-native execution for hybrid classical-quantum workflows. The natural emergence of these hybrid workflows implies a design in which it is necessary to identify relevant aspects of Quality of Service (QoS) by achieving a good balance among the non-functional properties of preexisting components or services, such as reliability, execution time, and cost. Consequently, it is necessary to analyze the trade-offs within the design space.

In this context, it is essential to consider the constraints and uncertainties of classical service components, as well as the additional constraints that naturally arise from integrating classical-quantum service-based systems. For instance, quantum services can only be deployed on specific hardware, come with different properties, and follow different pricing models. Moreover, these services introduce additional sources of uncertainty, such as the non-deterministic output of quantum circuits. Therefore, it is necessary to develop new tools, methodologies, or frameworks that facilitate the correct distribution and coordination of services across the IT infrastructure to ensure the desired quality of service, which can be very critical, as in the case of software systems.

Recent scientific work has begun to address these challenges. Cranganore et al. [23] propose hybrid cloud architectures for scientific workflows, discussing scheduling and resource allocation trade-offs; O’Riordan et al. [63] explore hybrid workflows for natural language processing; and Sivarajah et al. [87] develop a retargetable compiler (*t/ket*) that facilitates hybrid execution by bridging quantum programs with heterogeneous backends. These contributions mark an important step forward, yet they also underscore how immature the current methodologies are and how much remains to be done to generalize these approaches beyond isolated use cases.

Building on these insights, in Table 3 we identify several forward-looking research challenges that will become increasingly relevant in the coming years, as hybrid classical-quantum systems move from experimental setups to production-level environments.

Table 3. Hybrid Classical-Quantum Challenges for SOQ.

#	Challenge	Description
HB.1	Designing and Implementing Hybrid Workflows.	Methods and tools to model, implement, and validate hybrid classical-quantum workflows effectively.
HB.2	Efficient Orchestration of Hybrid Workflows.	Dynamic and fault-tolerant orchestration mechanisms for hybrid computations.
HB.3	Architectural and Data Model Compatibility.	Ensuring consistent and interoperable architectures and data formats across quantum and classical components.
HB.4	Quality of Service Management in Hybrid Environments.	Defining and enforcing QoS metrics and SLAs specific to hybrid classical-quantum workflows.

HB.1 Designing and Implementing Hybrid Workflows

Designing and implementing hybrid workflows is a foundational challenge for hybrid classical-quantum computing. Current implementations are often handcrafted, domain-specific, and lack formal design patterns or reusable templates. This hinders scalability and makes it difficult to adapt workflows to different contexts or hardware. To overcome this, the community needs to develop systematic methodologies and

domain-specific languages that allow engineers to express hybrid workflows at a high level of abstraction, along with hybrid-aware validation and testing strategies.

Steps forward include defining formal workflow models, creating repositories of reusable hybrid templates, and integrating hybrid workflows into continuous integration and deployment pipelines to facilitate iterative development and testing.

One notable example is in climate modeling and weather forecasting, where hybrid workflows are already proving valuable. Since these simulations require vast computational power and information obtained from classical data sources, managing when and how computational infrastructure is utilized is crucial to avoid bottlenecks and ensure critical computations are completed within tight time constraints. Google's Analog-Digital Hybrid Quantum Simulation⁹ has demonstrated how quantum subroutines can enhance long-term climate predictions when orchestrated within large classical simulation pipelines. In such contexts, designing reusable and verifiable hybrid workflows is critical to ensuring reliable predictions that can inform disaster response planning.

HB.2 Efficient Orchestration of Hybrid Workflows

Efficient orchestration is critical for dynamic allocation of tasks between quantum and classical resources in a way that minimizes latency, maximizes throughput, and maintains fault tolerance. Future research should focus on designing hybrid orchestration policies that explicitly consider performance uncertainty and resource heterogeneity.

Promising steps include using AI-based schedulers to predict quantum back-end availability, adding mechanisms to orchestrate classical and quantum parts, and developing hybrid workflow engines that optimize task allocation in real time.

BMW¹⁰ and Airbus¹¹ provides an illustrative case, that are experimenting with hybrid workflows for aerodynamics optimization and supply chain logistics. In these contexts, orchestrators must determine how and when to offload computationally expensive subproblems to quantum processors while ensuring that the classic workflow structure remains efficient and synchronized.

HB.3 Architectural and Data Model Compatibility

The incompatibility of the architectural and data model between quantum and classical components remains a significant barrier to seamless integration. Classical systems use deterministic floating-point representations, while quantum results are inherently probabilistic and often represented as amplitudes or density matrices. Future work should prioritize the development of standard intermediate representations and hybrid architecture reference models, along with automated translators that convert quantum outputs into formats that classical components can process efficiently.

An illustrative example is in hybrid molecular dynamics simulations [44], where quantum electronic structure calculations produce data that must be integrated into classical force-field models for biomolecular interactions. Bridging this gap effectively could accelerate drug discovery pipelines by making hybrid simulations more robust and easier to implement across platforms. Moreover, IoT edge devices collect data continuously, and quantum solvers in the cloud can optimize routes and inventories [65]. Determining the optimal balance between local pre-processing at the edge and remote quantum

⁹IEEE Spectrum (2023). Google's Analog-Digital Hybrid Quantum Simulation. Retrieved from <https://spectrum.ieee.org/quantum-simulation>

¹⁰BMW Group (2025). Quantum Computing News. Retrieved from <https://www.bmwgroup.com/en/news/general/2025/quantum-computing.html>

¹¹Airbus (2024). Airbus Quantum Computing Challenge. Retrieved from <https://www.airbus.com/en/innovation/digital-transformation/quantum-technologies/airbus-and-bmw-quantum-computing-challenge>

computation is critical to ensuring timely, reliable decisions.

HB.4 Quality of Service Management in Hybrid Environments

Finally, managing QoS in hybrid workflows is a critical but complex challenge, arising from the fundamentally different performance, reliability, and cost profiles of quantum and classical services. Classical components are generally predictable, with mature monitoring and SLA mechanisms, whereas quantum components exhibit stochastic behavior, variable execution times, and hardware-specific error rates that can fluctuate depending on calibration and environmental conditions. This asymmetry creates difficulties in defining, negotiating, and enforcing meaningful QoS guarantees for hybrid applications.

Addressing this challenge requires several coordinated efforts. First, hybrid-specific QoS metrics must be defined to accurately capture the interplay between classical throughput and quantum probabilistic fidelity. Such metrics should reflect not only standard properties like response time and availability, but also quantum-specific aspects such as success probability, decoherence impact, and number of repetitions (shots) required to achieve statistical significance. Second, SLA negotiation protocols need to evolve to account for hybrid dependencies, allowing contracts to specify acceptable ranges of performance that consider quantum uncertainty. Finally, real-time hybrid monitoring infrastructures must be developed to continuously track and analyze QoS indicators across the heterogeneous environment, detecting anomalies and providing actionable feedback to orchestrators and end-users.

The impact of this challenge is evident in domains where regulatory or mission-critical constraints demand consistent performance. Financial risk analysis systems, for example, depend on predictable response times and accuracy for compliance reporting. Hybrid Monte Carlo simulations¹² that combine classical and quantum resources must meet strict SLAs while balancing cost and resource utilization, requiring hybrid-aware SLA management to avoid regulatory breaches or costly re-runs.

5.5 Continuous Changes

Quantum services are undergoing continuous evolution, characterized by frequent modifications in pricing models, availability, and feature sets. This dynamic nature poses significant challenges for the long-term planning, contracting, and orchestration of quantum services. Cloud providers frequently introduce new QPUs, necessitating continuous updates in service compositions to ensure compatibility and optimal performance. A notable example is Amazon Braket, which grants developers access to different quantum computing technologies from multiple hardware providers, such as IonQ, D-Wave, or Rigetti. However, due to the constant evolution of the quantum ecosystem, certain machines become unavailable over time. In addition, some services are no longer available directly from quantum providers, but are instead offered through alternative services, as seen in the case of D-Wave¹³, where customers can no longer access the D-Wave 2000Q and Advantage systems through Amazon Braket.

Similarly, IBM Quantum has introduced a series of advancements in its quantum computing services¹⁴, including the deployment of new quantum machines and frequent updates to its quantum programming language. These continuous modifications add further complexity to the development and maintenance of quantum service-oriented architectures, requiring adaptive solutions for interoperability and orchestration. In this context, techniques for monitoring service quality and ensuring seamless transitions between quantum hardware generations are crucial to maintaining service reliability and usability. Therefore, without standardized classical-quantum

¹²Google Research (2022). Hybrid Quantum Algorithms for Quantum Monte Carlo. Retrieved from <https://research.google/blog/hybrid-quantum-algorithms-for-quantum-monte-carlo>

¹³AWS Quantum Technologies Blog (2022). Using D-Wave Leap from the AWS Marketplace. Retrieved from <https://aws.amazon.com/es/qc-dwave-marketplace>

¹⁴IBM Quantum Roadmap. Retrieved from <https://www.ibm.com/roadmaps/quantum>

communication protocols and the continuous changes, the integration of quantum solvers into existing financial platforms would require recurrent customizations. Table 4 summarizes some of these challenges that have been identified.

Table 4. Continuous Changes Challenges for SOQ.

#	Challenge	Description
CC.1	Quantum SDK Control Version.	Tool to facilitate the migration of different providers' SDKs.
CC.2	Dynamic Resource Allocation.	Quantum computing efficiency relies on “liquid” applications that adapt across the classical-quantum continuum, optimizing resources and accelerating practical solutions.
CC.3	Updating Job in Real-Time.	Quantum software development is hampered by long job execution waiting times due to scarce resources and maintenance, compounded by the critical inability to modify circuits once queued, severely hindering agile progress.

CC.1 Quantum SDK Control Version

The rapid advancement and fierce competition among quantum computer providers lead to frequent and often unannounced updates in their Software Development Kits (SDKs). These updates, driven by continuous innovation, frequently introduce significant improvements, new functionalities, and simultaneously, deprecated functions or altered APIs. While essential for pushing the boundaries of quantum computing, this constant evolution presents a substantial hurdle for software developers. The inherent instability can cause significant incompatibility errors in developed quantum software, particularly during critical phases such as transpilation, optimization, and execution on diverse quantum hardware back-ends. This dynamic environment poses a considerable challenge for developing stable, reliable, and maintainable quantum solutions that can endure over time without constant manual intervention.

To effectively address this critical challenge, it's imperative to develop sophisticated, automated tools and methodologies. These tools must be capable of intelligently identifying, analyzing, and automatically updating quantum software to the latest SDK versions. Crucially, such systems should incorporate robust automated testing frameworks designed to proactively detect any incompatibility issues or performance regressions within the production codebase immediately following an SDK update. If the automated verification process identifies errors, the system's output should be highly granular and actionable, clearly indicating the specific failures, their root causes, and their precise locations within the code, facilitating rapid debugging and remediation. Conversely, upon successful verification, the system should enable an automated, seamless deployment of the newly updated service. This automated deployment should strategically manage the replacement of existing instances running on older service versions with the newly validated ones, ensuring minimal disruption and continuous service availability. This proactive approach is vital for fostering agility and resilience in the face of rapidly evolving quantum hardware and software landscapes.

CC.2 Dynamic Resource Allocation

Dynamic resource allocation in quantum computing remains a major challenge due to the limited availability and highly specialized nature of quantum hardware. In contrast to classical computing, where resources are generally abundant and easily interchangeable, QPUs are scarce, expensive, and exhibit unique characteristics that influence algorithmic performance. This limitation, combined with long queue

times and the heterogeneous capabilities of existing QPUs, makes efficient resource utilization essential but particularly difficult. Within this complex context, the concept of liquidity emerges as a key enabler. By giving both classical and quantum software components liquid-like properties, it becomes possible to achieve the flexibility required for applications to dynamically adapt and operate across the classical-quantum continuum. In this way, resources can be optimally leveraged at any given moment to ensure efficient execution and maximize throughput.

This challenge can be addressed through the development of adaptive classical-quantum orchestration layers that go beyond basic job scheduling. Such layers would analyze QPU availability, workload conditions, and specific job requirements in an intelligent manner. At the same time, the use of fine-grained quantum microservices is recommended, allowing complex algorithms to be divided into smaller tasks that can be executed on the most suitable QPU, or even on classical resources when immediate quantum access is unavailable. Applications should also be designed to support real-time shifting within the classical-quantum continuum, enabling them to dynamically reroute or approximate computations based on current resource conditions. In addition, proactive resource monitoring and prediction would help anticipate future QPU states, improving job placement decisions. Finally, automated quantum program transpilation and optimization for different hardware targets would ensure efficient execution regardless of the specific QPU selected.

By applying these strategies, quantum applications would no longer be constrained by fixed hardware or long waiting times. Instead, they would gain liquidity, seamlessly flowing across the classical-quantum continuum, adapting dynamically to resource availability, and accelerating the development and deployment of practical quantum solutions.

CC.3 Updating Jobs in Real-Time

Quantum software development grapples with a significant bottleneck, the protracted waiting periods for job execution. This delay is primarily driven by the high demand for scarce quantum computing resources from researchers across academia and industry, compounded by necessary maintenance windows for the quantum computers themselves. Consequently, it's not uncommon for a quantum job to experience execution delays of several days. While efforts are underway to maximize quantum machine utilization through techniques like simultaneous circuit execution [74], an additional challenge arises: the inability to modify a quantum circuit once it has been submitted to the queue.

This lack of in-queue modifiability presents several critical issues. For instance, if an urgent software update becomes available, perhaps addressing a newly discovered bug or improving an algorithm's efficiency, developers are currently unable to apply these changes to jobs already awaiting execution. The inability to implement such dynamic changes without entirely resubmitting a job exacerbates the already long waiting times and hampers agile development and iterative refinement of quantum software. Addressing this limitation is crucial for improving developers' productivity and accelerating quantum research.

5.6 Pricing Configuration Space

The design and management of pricing strategies in SOQ systems present novel and underexplored challenges, distinct from those in classical cloud computing. Unlike traditional SaaS environments, where pricing can be based on well-understood metrics such as CPU hours, storage, or bandwidth, quantum computing introduces pricing variables tied to the probabilistic and hardware-dependent nature of quantum executions. These include factors such as the number of shots, qubit coherence time, gate fidelity, error rates, execution priority, etc.

This complexity gives rise to a **multidimensional pricing configuration space** in SOQ, where pricing is not only a business model issue, but a core technical challenge that affects orchestration, scheduling, and service-level guarantees. In Table 5, we identify four main challenges in this context.

Table 5. Pricing Configuration Space Challenges for SOQ.

#	Challenge	Description
PCs.1	Multidimensional and Hardware-Dependent Pricing Models.	Quantum pricing depends on heterogeneous hardware back-ends with different fidelities, queue times, and guarantees, requiring dynamic, provider-specific models that jointly optimize cost, quality, and availability.
PCs.2	Hybrid Pricing in Classical-Quantum Workflows.	Hybrid workflows combine asymmetric cost models (time-based for classical vs. shot-based for quantum), demanding unified pricing schemes that balance fidelity, cost, and performance.
PCs.3	Dynamic Access and Priority-Based Pricing.	Scarcity of quantum hardware drives priority-based and pay-for-priority tiers, introducing temporal volatility into costs and limiting flexibility once jobs are queued.
PCs.4	Configuration Complexity and Economic Feasibility.	Complex pricing options create overwhelming configuration spaces, requiring automation and optimization tools to ensure economic feasibility.

PCs.1 Multidimensional and Hardware-Dependent Pricing Models

Quantum services are executed on heterogeneous hardware back-ends (QPUs), each with specific physical characteristics that influence both cost and performance. Providers like IBM, IonQ or Rigetti expose devices with varying fidelities, queue times, and execution constraints. As a result, a single quantum task may incur different costs depending on where and when it is executed. This creates the need for **pricing models that are dynamic, per-provider, and sensitive to quantum hardware properties**. Furthermore, advanced pricing tiers may offer guarantees on error thresholds or availability windows, further complicating the pricing space. From a service binding perspective, this challenge directly intersects with the *QoS-aware binding/composition problem* in service-oriented architectures [20]. Traditionally, binding a task to a provider involves evaluating a solution space defined by static QoS attributes. In SOQ with pricings, however, each provider introduces a set of pricing-dependent configurations, e.g., shot count limits, fidelity guarantees, or priority access, that effectively multiply the number of feasible bindings. Consequently, the solution space for the binding problem expands exponentially [35], requiring new approaches to evaluate and rank execution plans that jointly optimize for cost, quality, and availability under pricing constraints.

PCs.2 Hybrid Pricing in Classical-Quantum Workflow

In SOQ, pricing must account for hybrid service compositions where quantum subroutines are embedded within larger classical workflows. While classical services are typically priced per compute hour or API call, quantum components are priced per shot or per circuit execution. The **asymmetric cost model** of hybrid services poses significant challenges for orchestrators and clients, who must balance fidelity, cost, and performance trade-offs in real time. Hybrid pricing models should allow combining different metrics (e.g., time-based for classical, shot-based for quantum) in a unified representation.

PCs.3 Dynamic Access and Priority-Based Pricing

Given the scarcity of quantum hardware, many providers implement **priority-based queueing mechanisms** or “pay-for-priority” tiers, where higher payment guarantees faster or more stable execution. This introduces temporal volatility into pricing. A task’s final cost may depend on queue times, cancellations, or noise levels at the moment of execution. Moreover, current systems lack mechanisms for **real-time modification of jobs once queued**, limiting the ability to adapt to pricing changes after submission.

PCs.4 Configuration Complexity and Economic Feasibility

Users must navigate a large space of configurable pricing options, shots, fidelity constraints, noise-aware rerouting, execution priority, SLA guarantees, which can quickly lead to **overwhelming configuration spaces** [35]. These options must be analyzed not only for correctness [36] but for **economic feasibility**, especially in recurring or mission-critical scenarios. Techniques from classical pricing automation (e.g., iPricing or constraint optimization) may be adapted to quantum systems to support automated pricing validation, recommendation, and optimization. In the context of SOQ, quantum-specific properties such as qubit count, noise levels, and number of shots must be treated as declarative, contract-aware parameters within the pricing and service description layers. These parameters directly affect both the cost and feasibility of a service invocation and must therefore be seamlessly integrated into SLA definitions and exposed as part of the service metadata. Unlike QSOC, where such constraints are often addressed by introducing auxiliary validation tasks or imperatively altering the workflow, SOQ aspires to a declarative approach that avoids polluting the business logic. This means that service composition and selection mechanisms should validate whether the quantum back-end can satisfy the resource and fidelity requirements of a given task at design or planning time, not during execution. By formalizing these parameters as part of the pricing configuration space, SOQ enables dynamic service negotiation, fallback mechanisms, and workflow optimization, while preserving architectural modularity and abstraction.

5.7 Workforce training

As quantum computing advances, the lack of skilled professionals with expertise in quantum algorithms, hybrid classical-quantum systems, and SOC integration presents a significant barrier to adoption [98]. Unlike classical computing, which has a well-established developer ecosystem with standardized tools and best practices, quantum computing is still in its early stages, requiring specialized knowledge. Training a workforce capable of developing, deploying, and maintaining quantum services is therefore critical to ensuring the success of the broader quantum software ecosystem [42].

Workforce training is crucial for interoperability, as developers must integrate quantum services with classical IT infrastructures. A lack of expertise in hybrid architectures can hinder quantum adoption. Similarly, platform independence depends on skilled professionals who can adapt applications across different hardware providers, avoiding vendor lock-in. Demand and capacity management also requires trained engineers to optimize workloads and schedule classical-quantum computations efficiently, preventing resource waste and high costs. Moreover, complexity management, continuous changes, and pricing models require expertise in quantum computing economics, hardware constraints, and evolving algorithms. Without ongoing training, scaling quantum applications beyond NISQ-era devices will be challenging. Investing in workforce development through education, industry collaboration, and standardized training is essential to ensure SOQ evolves, integrates into enterprises, and delivers real-world benefits.

According to these insights, in Table 6 we identify several relevant challenges in workforce training development.

Table 6. Workforce Training Challenges for SOQ.

#	Challenge	Description
WT.1	Shortage of qualified professionals.	The current workforce lacks sufficient individuals with the specialized expertise required in quantum computing.
WT.2	Immaturity of the field and technical complexity.	Early stage of development of the field, with high technical complexity that hinders widespread understanding.
WT.3	Inherently interdisciplinary nature of quantum development.	Effective work in quantum computing demands integrated knowledge across physics, computer science, engineering, and mathematics.
WT.4	Insufficient or inadequate educational offerings.	Educational programs in quantum computing are limited in availability and scope.

WT.1 Shortage of qualified professionals and a low critical mass of researchers

There is a significant scarcity of professionals with specialized knowledge in Quantum Technologies, particularly in the area of Quantum Computing. To address the shifts of this new paradigm, an increase in personnel with this expertise is required. Currently, there is still a low critical mass of researchers in QSE, which underscores the urgent need to train new researchers and students in the specific aspects of this field. Furthermore, the demand for qualified professionals in quantum software development is growing rapidly, creating an urgent need to fill job vacancies in the coming years.

WT.2 Immaturity of the field and technical complexity

Current quantum computers are noisy and susceptible to errors, which limits their practical utility for complex computations and necessitates sophisticated error correction techniques to mitigate these problems. Likewise, many of the elements of the infrastructure, tools, and techniques needed to develop quantum software solutions are in their early stages. This implies that training must not only address advanced theoretical concepts but also the practical limitations and challenges of current technology. The lack of standards also affects both technical progress and commercial adoption. This hinders interoperability between platforms, programming languages, and hardware, scalability, i.e., difficulty in expanding capabilities and efficiently connecting multiple systems, and collaborative innovation among scientific, academic, and industrial communities. The absence of standards in quantum computing creates unnecessary complexity for learners, discourages entry-level participation, and hinders the development of cohesive educational programs. As the field matures, establishing educational standards and cross-platform compatibility will be essential for building a broad, skilled quantum workforce.

Another key factor that limits the opportunities for learning quantum computing is the high cost of accessing quantum infrastructure. It significantly limits the democratization of knowledge and practical training opportunities in quantum systems education, while also amplifying inequalities between institutions worldwide. To make quantum education more inclusive and sustainable, it is essential to develop supported access models, shared affordable infrastructure, advanced free or open-source simulators, and promote strong partnerships between industry, government, and academia.

WT.3 Inherently interdisciplinary nature of quantum development

Quantum development requires fundamental contributions from mathematicians and physicists, demanding collaborative approaches for its advancement. Future professionals must possess a foundational

theoretical understanding of quantum physics principles (fundamental concepts, mathematical frameworks, qubit dynamics) and the underlying physical principles of quantum technologies. Additionally, they need advanced expertise in a specific aspect of quantum technology and a keen understanding of the interrelationships among various facets of quantum technology and classical systems, including the integration of hybrid classical-quantum systems.

WT.4 Insufficient or inadequate educational offerings

Although the academic offerings in quantum computing are growing, with courses and master's degrees provided by various platforms (such as EdX or Udemy) and organizations (like IBM, Google, and Microsoft), most of these offerings focus on general introductory aspects or deal with quantum computing at a low level. There is a relative scarcity in the educational offerings that address the more specific and advanced aspects of quantum computing. For the implementation of this technology in productive sectors, it is crucial to increase the number of qualified individuals who can participate in research and development projects in the field.

Therefore, to build a professional profile, it is essential to understand the professional requirements established by experts and companies, such as those outlined in the European Competence Framework for Quantum Technologies (ECFQT). These requirements not only cover a solid theoretical foundation and advanced expertise in specific areas but also advanced technical skills such as understanding quantum computing principles, qubits, quantum gates, circuits, and algorithms (like Shor or Grover), quantum hardware technologies, quantum programming languages and SDKs, quantum error correction, quantum circuit optimization, and quantum software applications in real-world use cases. Furthermore, the need for practical experience through laboratories, simulations, and applied projects is emphasized, along with "soft skills".

In summary, the main challenge is to bridge the gap between the increasing demand for specialized talent in QSE and the limited supply of educational programs that provide the necessary depth and practical focus for such a complex and interdisciplinary field. Many initiatives, educational centers, and universities have detected this need and have begun to offer specialized courses and master's degrees in this field. A clear example is the role of the *European initiative Quantum Flagship* [73], which among its objectives, is the training and development of talent in the quantum field. Specifically, the initiative will promote various programs to educate and train the next generation of scientists, engineers, and professionals who will work in this emerging industry. DigiQ (Digitally Enhanced Quantum Technology Master) is the primary workforce development project of the Quantum Flagship. It is funded by a Euro 17.6 million grant over four years through the European Commission's Digital Europe Programme. There are industrial associations specifically dedicated to promoting the use and adoption of quantum computing globally. These organizations bring together companies, universities, startups, and governments to collaborate on growing the quantum ecosystem. For instance, the *European Quantum Industry Consortium* (QuIC). The aim of this association is to accelerate the industrial development of quantum technologies in Europe. They also work in collaboration with the Quantum Flagship initiative.

6 Discussion

The preceding analysis of challenges reveals a complex ecosystem where engineering, economics, and human capacity co-evolve. Despite being presented in separate categories, most challenges are deeply interdependent. This section presents a discussion that synthesizes cross-domain areas and highlights research opportunities offered by the interaction between challenges.

Interoperability and Platform Independence challenges highlight the urgent need for common abstractions and portable service execution models. IP.1–IP.4 demand standardized service interfaces, deployment tooling, data models, and adaptive orchestrators. These are prerequisites for eliminating vendor lock-in and for enabling “liquid quantum services” that seamlessly move across providers.

Demand and Capacity Management challenges (DC.1–DC.4) expose the lack of formal mechanisms to declare user intent, forecast execution performance, and synthesize executable plans. They bridge interoperability with economics: service execution cannot be abstracted from queue latency, probabilistic quality, or provider heterogeneity. Adaptive scheduling, demand-aware orchestration, and predictive models become essential.

Hybridity challenges reveal that quantum computing does not replace classical infrastructure, but rather integrates with it. HB.1–HB.4 reveal this architectural mismatch. For example, quantum circuits, noise profiles, density matrices, and probabilistic outcomes diverge from classical deterministic data models. As a result, orchestration, quality of service, and workflow semantics cannot be defined independently of hardware constraints without reusable hybrid patterns, intermediate representations, and validation pipelines.

Continuous Changes challenges capture the volatility of the technological ecosystem. CC.1–CC.3 show how evolving SDKs, limited hardware availability, and long queue times demand runtime adaptation, automatic refactoring, live job modification, and hardware-aware liquidity. These conditions place continuous stress on cost models, interoperability promises, and QoS guarantees, turning static planning frameworks obsolete.

Pricing Configuration space consists of transversal challenges (PC.1–PC.4) that influence almost all of SOQ’s operational decisions. The pricing landscape is multidimensional and depends on the hardware and the quantum service provider. Specifically, providers charge per shot, per execution time, per priority level, or through composite packages that include fidelity guarantees or reserved access windows. This multiplicity produces a wide variety of configurations when combined with quality of service constraints, heterogeneous back-ends, hybrid classical-quantum workflows, and input-dependent performance regimes. Therefore, pricing affects binding decisions, coordination strategies, and plan synthesis, shaping the boundary at which the use of quantum resources becomes economically and operationally justified [64]. Addressing the pricing configuration space thus requires new abstractions, decision algorithms that jointly optimize cost and fidelity across hybrid resources, and governance primitives that enable pricing-aware decision-making, formal reasoning about such decisions, and auditable economic transactions between providers.

Finally, **Workforce Training** challenges (WT.1–WT.4) demonstrate an immaturity of tooling, interdisciplinary barriers, and inadequate educational offerings. Without a trained workforce, technical advances remain theoretical. Therefore, without standards and shared infrastructure, educational institutions cannot scale training or democratize access.

The interaction of these challenges leads to interoperability standards influencing coordination, coordination determining price feasibility, prices affecting hybrid workflow design, hybridization interacting with SDK evolution, and all of this being constrained by workforce capacity. These interactions therefore define different research opportunities.

Table 7 serves as a summary of the research opportunities generated by the challenges, identifying them as primary and secondary challenges.

This traceability exposes the fact that the future of SOQ cannot be driven by isolated technical advances. These research opportunities arise where cross-challenge intersections occur, such as standard interoperability

Table 7. Research Opportunities and their relation to SOQ challenges.

Research Opportunity	Description	Primary Challenges Addressed	Related Secondary Challenges
Hybrid quantum service interfaces and metadata schemes	Define standardized, cross-provider service interfaces and metadata schemes capturing hybrid execution requirements, probabilistic outputs, noise descriptors, and pricing/QoS constraints.	IP.1 IP.3 HB.3	IP.2 IP.4 DC.1 HB.4 WT.2
Adaptive Hybrid Orchestration and Plan Synthesis	Create orchestration engines that adjust plans at runtime using telemetry, noise forecasts, SDK version changes, and hybrid workflow requirements.	DC.3 IP.4 HB.2 CC.2	DC.4 HB.1 HB.4 CC.1 CC.3
Cross-Backend Deployment and Adaptative Mechanisms for Quantum Services	Design mechanisms that enable seamless movement of services across hardware back-ends, including the co-design of real-time transpilation scheduling and job migration.	IP.2 DC.4 CC.2	IP.1 IP.3 CC.3 HB.1 PCs.4
Intermediate Representations and Validation Pipelines for Hybrid Applications	Create IRs and automated validation tools that bridge classical and quantum components, supporting error modeling, semantic preservation, and cross-SDK compatibility.	HB.1 HB.3 CC.1	IP.3 DC.1 DC.3 WT.2 WT.3
Pricing-Aware Hybrid Classical-Quantum Service Orchestration and Economic Optimization Models	Develop algorithms and formal models that jointly optimize fidelity, latency, cost, and priority-based pricing across hybrid workflows, enabling pricing-aware decision-making, input-dependent binding selection and resource allocation under dynamic queue and hardware conditions.	PCs.1 PCs.2 PCs.3 PCs.4	DC.2 DC.4 IP.4 CC.2 CC.3
Education, Tooling Infrastructure, and Skills Standardization for Quantum Software Engineering	Establish educational frameworks, competency models, and tooling standards to reduce the skills gap and enable interdisciplinary training at scale.	WT.1 WT.3 WT.4	WT.2 IP.1 HB.1 HB.2 CC.1

schemes coupled with QoS semantics, hybrid orchestration, pricing models informed by noise and liquidity, and educational frameworks aligned with real deployment constraints.

7 From Adaptation to Native Integration: SOQ vs. QSOC

This section highlights the main differences between SOQ and QSOC by analyzing the current technology stack for QSOC and proposing the new layers for SOQ. First, grounding the discussion in a concrete classical service implementation, and then using it as a reference point to define a layered architecture for quantum services.

7.1 A “Good” Classical Service Implementation

To illustrate the foundational principles of Service-Oriented Computing, Fig. 3 depicts a representative implementation of a classical service designed according to well-established SOC practices. This example reflects key architectural properties such as interface-based interaction, loose coupling, and modular encapsulation, all of which are essential for enabling interoperability, reusability, and dynamic composition in distributed environments.

Although there is no single “best” way to implement a classical service, decades of industrial experience have converged on a set of recurring architectural patterns and standards that enable scalable, maintainable, and interoperable services. The Figure captures these dominant practices. Therefore, it distinguishes three phases of the service life-cycle: implementation, execution and deployment, and monitoring.

During the **implementation phase**, service developers combine ad-hoc business logic with a formally specified service interface. Standards such as OpenAPI or gRPC allow the service interface to be described in a language-agnostic manner, enabling automated client and server stub generation. This separation allows the business logic to evolve independently from interface concerns and supports interoperability across programming languages and platforms.

In the **execution and deployment phase**, services are deployed on cloud infrastructure using a range of models, from virtual machines to containers or elastic compute clouds. Here, there are different Platforms as Services (PaaS), including IBM Cloud, AWS, Azure, or App Engine to perform the execution and deployment of the service. Moreover, for the service integration, API Gateways typically mediate all incoming requests, routing them to the appropriate service instances, and shielding clients from deployment details. Load balancers and elastic scaling mechanisms ensure that services can handle variable demand while maintaining predictable behavior. This phase also takes into account CI/CD techniques that serve to automate the entire software life-cycle (creation, testing, version control, and deployment) using DevOps tools such as GitHub Actions, enabling faster, more frequent, and more reliable releases, improving code quality, detecting errors early, and shortening feedback loops, all while managing version control to maintain a clear history and be able to revert changes.

Finally, in the **monitoring phase**, the operational tools collect logs, metrics, and usage data. These mechanisms enable developers to observe service behavior, detect anomalies, control operational costs, and enforce service-level objectives. Importantly, monitoring is not an afterthought, but an integral part of the service architecture.

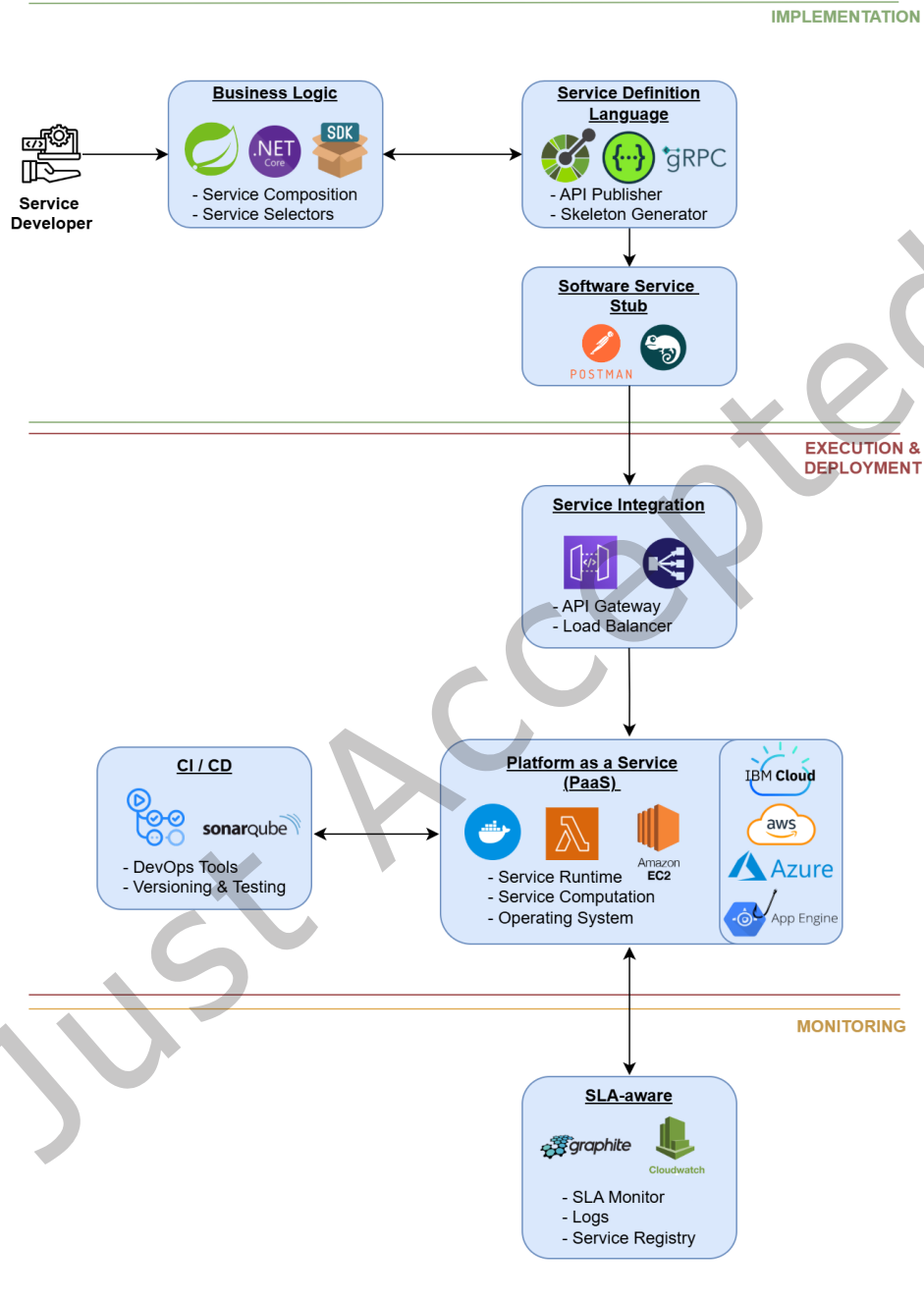
This classical architecture demonstrates that even relatively simple services rely on a rich ecosystem of standards, tools, and platforms. These elements collectively enable professional software engineering practices and serve as a baseline for evaluating whether similar benefits can be achieved for quantum services.

7.2 QSOC Technology Stack

The modern computing landscape is the result of over six decades of technological evolution, refinement, and standardization. From the early days of transistor-based machines to today’s globally distributed cloud systems, this layered stack of technologies forms the foundation upon which all software, ranging from web applications to mission-critical systems, has been built. The gradual stratification into layers has enabled modularity, portability, scalability, and increasingly abstracted interaction with underlying hardware.

At the lowest level, we find the hardware layer, which has evolved from vacuum tubes and early mainframes to powerful microprocessors, GPUs, and accelerators like FPGAs and TPUs. These components are responsible

Fig. 3. Architecture of a “good” classical service implementation.



for the fundamental execution of machine-level instructions and are manufactured to support general-purpose, high-performance, or domain-specific computing.

On top of this sits the operating system (OS) layer, responsible for abstracting hardware complexity and providing interfaces for process management, memory allocation, I/O operations, and user-level execution. Operating systems like Linux, Windows, and macOS have become the dominant platforms in personal, server, and cloud computing environments, with specific variants for real-time or embedded systems.

With the rise of virtualization and lightweight computing, the container layer emerged, most notably through technologies like Docker. Containers offer isolated execution environments that are portable and fast to deploy, becoming central to microservices architectures. This was followed by container runtimes like containerd and CRI-O, which provide the core interfaces to manage container lifecycles.

To scale containerized applications, the orchestration layer became essential. Technologies such as Kubernetes allow for the management of thousands of containers across clusters of machines, offering high availability, load balancing, autoscaling, and fault tolerance. This layer transformed how distributed systems are deployed and maintained, introducing the “infrastructure as code” mindset.

Above this sits the cloud infrastructure layer, provided by global hyperscalers such as AWS, Microsoft Azure, and Google Cloud Platform. These platforms offer elastic computing, storage, and networking resources as services, allowing developers to focus on building applications without worrying about physical infrastructure. Concepts like pay-as-you-go billing, serverless computing, and global scalability are native to this layer.

On top of these foundational components, we find the service and application layer, which enables cloud computing, SOA, and microservices. Applications are decomposed into independent services that communicate through REST APIs, gRPC, or event-driven messaging systems. These services are developed using a range of programming languages and frameworks, such as Python, Java, Go or Node.js, and integrated into business workflows using orchestration tools like Apache Camel, Istio, or workflow engines.

At the same level as the cloud, we locate QSOC. While still rooted in classical infrastructure, QSOC represents the beginning of an architectural integration between classical service-oriented systems and quantum services. In QSOC, quantum resources are treated as remote services, invoked via classical workflows, without requiring the programmer to manage the complexity of quantum hardware directly. These services are often exposed through cloud APIs (e.g., AWS Braket or IBM Quantum) and are tightly coupled with classical layers for orchestration, preprocessing, and post-analysis.

This layered model, from hardware to QSOC, has enabled the modular, scalable, and resilient systems we rely on today. Each layer abstracts and builds upon the one beneath it, allowing innovation to flourish independently while maintaining interoperability and reliability.

7.3 Proposed SOQ Layered Technology Stack

The emerging field of quantum computing is progressing rapidly, but it remains in an early stage of development compared to the mature, multi-layered stack of classical computing, as we saw in the previous section. While classical systems continue to evolve at a rapid speed, quantum computing still relies on highly specialized hardware, fragmented software stacks, and vendor-specific execution environments.

With the SOQ paradigm, which we propose in this paper, we want to anticipate a future in which quantum computing can be abstracted, composed, and integrated as a native part of service-oriented architectures. However, for SOQ to become a reality, we believe it is necessary to develop a robust, layered quantum technology stack. This stack must provide not only access to quantum capabilities, but also interoperability, automation, pricing models, and service governance, similar to what classical stacks offer today. We understand that this transition must be gradual, but that the idea must be focused so that the entire industry and scientific community work along the same lines. For these reasons, we propose a layered model indicating existing quantum technology, the

limitations that exist in each of the layers, and the requirements for achieving SOQ. To summarize, we provide the next Fig. 4 showing the proposed layers for SOQ, the current shortcomings, and the progress needed to achieve SOQ.

Fig. 4. Summary of Quantum Technology Stack: Deficiencies and Research Priorities.

Layer	Current technologies	Focus for Advancement	Deficiencies
Quantum hardware	D-Wave, AWS, IBM Quantum	Open APIs, standard benchmarking, modular hardware interfaces	Proprietary access, no common metrics, platform heterogeneity
Quantum OS / Runtime	Qiskit Runtime, FIRE OPAL, Amazon Braket	Shared runtimes, resource abstraction, unified execution layers	No general-purpose OS, limited scheduling and multi-tenancy
Programming / SDKs	Q#, Qiskit, Cirq	Intermediate representations (e.g., QIR), portable compilers, SDK standardization	Vendor lock-in, lack of interoperability, no unified IR
Hybrid Orchestration	—	Hybrid-aware orchestration engines, declarative and portable workflows	Ad-hoc integration, dependence on cloud providers
Service Abstraction	OPENAPI *	Formal service definitions, reusable APIs, service discovery and integration	Minimal APIs, no service composition or registry standards
Governance	—	SLA-aware service contracts, pricing metadata, cost-based scheduling	Basic pricing models, lack of SLAs, opaque cost-performance mapping
Ecosystem	Github Actions, sonarqube *	Quantum DevOps tools, testing frameworks, versioning and deployment pipelines	No CI/CD, poor testability, no reproducibility or modularity

 **Partially**
 **No technology that explicitly address**

- (1) **Quantum Hardware Layer.** At the base of the stack lies the quantum hardware layer, composed of diverse technologies such as superconducting qubits (IBM, Google), trapped ions (IonQ, Quantinuum), photonic qubits (Xanadu), and neutral atoms (Pasqal). Perhaps this layer is the most advanced and the one in which the greatest efforts and investments are being made. Although these technologies differ in coherence

time, qubit connectivity, gate fidelity, and scalability. While they have enabled promising demonstrations of quantum advantage and commercial access, each vendor maintains proprietary hardware interfaces, performance, operability, and metrics, which limit interoperability and comparative benchmarking. To build a sustainable quantum ecosystem, the industry must move toward standardizing technology and developing more modular and interchangeable hardware interfaces, similar to what has been achieved with CPUs and GPUs in classical computing.

- (2) **Quantum Operating System and Runtime Layer.** Located at the top of the hardware stack, the control and runtime layer governs the low-level execution of quantum operations. There are already some offerings in this layer that include vendor-specific firmware and early runtime environments such as Qiskit Runtime (IBM), Cirq Runtime (Google), Amazon Braket (AWS), or FireOpal (Q-CTRL), which handle pulse-level control, job submission, and error mitigation. However, the quantum field still lacks a general-purpose operating system capable of scheduling processes, isolating workloads, or virtualizing resources. Moreover, unlike the classical domain where POSIX has provided a widely adopted standard for interoperability and abstraction, there is currently no equivalent framework guiding the design of quantum operating systems. This absence of a unifying standard hinders the development of robust multi-user environments and complicates integration with high-level orchestration tools. Future quantum systems will require shared runtimes, portable low-level APIs, and unified control layers that can manage execution across diverse devices in a predictable and scalable manner.
- (3) **Programming and SDK Layer.** The programming layer allows developers to write and test quantum applications using high-level languages and SDKs. Popular frameworks such as Qiskit, Cirq, PennyLane, and Q# have democratized quantum programming by enabling circuit design, hybrid algorithm development, and integration with classical logic. Despite this progress, most SDKs are tightly coupled to their underlying (classical) platforms, limiting cross-vendor compatibility and confining developers to specific ecosystems. Furthermore, the absence of a universally adopted intermediate representation hinders code portability and toolchain interoperability. Research and development should focus on defining common IRs (e.g., QIR), improving compiler portability, and creating toolchains that allow quantum programs to run on different hardware back-ends without rewriting or vendor-specific tuning. These efforts align with the emerging field of Quantum Software Engineering, which emphasizes proper abstraction mechanisms, enhanced expressiveness of quantum programs, and greater developer productivity, thereby complementing the technical advances in interoperability and portability [68].
- (4) **Hybrid Classical-Quantum Orchestration Layer.** Since current quantum devices are still in the NISQ era, most useful algorithms are based on hybrid execution models. Platforms such as Amazon Braket and IBM provide hybrid task orchestration that allows classical systems to coordinate the execution of quantum circuits. However, these approaches remain limited because they are embedded within provider-specific runtimes, lack standardized workflow representations, and do not support multi-provider scheduling or portable hybrid pipelines. The absence of orchestration technologies stems from the lack of stable hardware performance, rapidly fluctuating noise profiles, and the absence of shared resource descriptors or intermediate representations that orchestration engines could target. As a result, existing orchestration remains ad hoc and cannot provide dynamic task placement, distributed coordination, or governance-aware scheduling comparable to mature classical workflow engines.
To enable scalable hybrid applications, the ecosystem must develop orchestration frameworks that are aware of quantum constraints such as noise, coherence time, and queue latency, and that are capable of intelligently allocating tasks at execution time between classical and quantum resources using flexible, declarative workflows. A key challenge is also to determine when quantum execution is truly advantageous compared to classical alternatives. Such decisions depend not only on runtime factors (e.g., noise, coherence,

invocation overheads) but also on governance and pricing models that influence cost-effectiveness and sustainability.

- (5) **Service Abstraction Layer.** A fundamental requirement of SOQ is the ability to expose quantum functionalities as services, with well-defined interfaces, standardized inputs and outputs, and service quality descriptors. Current platforms, such as IBM Quantum and AWS Braket, offer basic service interfaces through REST APIs or SDKs, but these are often limited to job submission and lack deeper composability features. Furthermore, there are no industry-wide conventions for describing quantum services, publishing them in registries, or integrating them into multiservice architectures. Advancing this layer will require formal service definition standards, composable APIs, service registries, and mechanisms for interface negotiation and semantic interoperability between heterogeneous quantum providers.
- (6) **Governance.** Unlike classical computing, where SLAs, pricing models, and quota systems are well established, quantum computing lacks unified approaches to economic governance. Current pricing schemes are basic and tied to metrics such as the number of shots, qubit usage, or execution priority. Technologically, governance remains undeveloped because providers do not offer performance guarantees, do not publish stable price-to-loyalty ratios, and do not offer mechanisms for negotiating SLAs. The variability and stochasticity of NISQ devices prevent providers from committing to predictable guarantees, and the absence of standard descriptors for cost, latency, or hardware reliability makes it difficult to formalize governance between different providers.
To launch large-scale quantum services, platforms must implement pricing metadata, SLA descriptors, and brokerage mechanisms that allow applications to select providers based on fidelity, execution time, and cost constraints. This also opens the door to cost-conscious orchestration and market-based resource optimization.
- (7) **Ecosystem Layer.** In classical software engineering, DevOps practices such as version control, CI/CD processes, automated testing, and observability are the norm. In quantum software, these practices are virtually absent. Development is often done using notebooks and scripts, with limited tools for testing quantum behavior, simulating realistic noise, or tracking quantum job history. Furthermore, there is no support for reproducibility, dependency management, or modular code reuse. For SOQ to be viable, the ecosystem must provide DevOps pipelines integrated into SDKs, domain-specific testing frameworks, and versioning and packaging tools compatible with quantum technology. Beyond adapting classical DevOps practices, the ecosystem will likely need inherently quantum-oriented tools, such as dashboards or monitoring frameworks that expose quantum-specific execution properties (e.g., qubit usage against resource limits, noise levels, or average shots per program). These native observability capabilities would complement traditional pipelines and provide developers with meaningful feedback about workload efficiency and performance. This combination will enable professional-grade software engineering practices in quantum development and lower adoption barriers for enterprises and open-source contributors.

7.4 A SOQ Reference Architecture

Taking into account the layers identified above and based on the classical reference architecture and its three main phases, a reference architecture for quantum services under the SOQ paradigm is now defined. Therefore, Fig. 5 illustrates this reference architecture, where the SOQ layers are encompassed in the different phases of implementation, execution and deployment, and monitoring of services.

In the implementation phase, SOQ envisions that quantum services are being developed using programming frameworks and SDKs that encapsulate quantum business logic, such as circuit construction or hybrid algorithms. Existing tools such as Qiskit, Cirq, PennyLane, or Q# partially fill this role. However, unlike the classical case,

there is no standardized service definition language for quantum endpoints, no widely adopted mechanism for API publication, with the exception of some research papers that make use of the OpenAPI Specification, and no tooling for generating quantum service stubs. As a result, quantum service interfaces remain implicit, vendor-specific, and tightly coupled to SDKs.

The execution and deployment phase reveals even more pronounced gaps. While cloud platforms such as IBM Quantum or Amazon Braket, as well as specialized vendors like D-Wave and Rigetti, provide access to quantum hardware, they expose quantum computation primarily as queued jobs rather than as deployable, long-lived services. This phase currently encompasses the hardware, operating system, and runtime layers, which are largely opaque to developers and tightly controlled by providers. As a consequence, developers have limited or no control over the execution environment, no standardized orchestration mechanisms, and only partial or ad hoc implementations of API gateways or load balancers capable of reasoning about quantum-specific constraints such as noise, queue latency, hardware calibration drift, or fidelity guarantees. The execution and deployment phase in SOQ also includes an ecosystem layer, which is responsible for DevOps-oriented activities such as CI/CD, automated testing, validation, and release management of quantum services. In contrast to classical service ecosystems, this layer remains largely immature in the quantum domain. There are currently no standardized tools, interfaces, or specifications to support quantum-aware CI/CD pipelines, or automated deployment strategies across heterogeneous quantum back-ends. Nevertheless, as explained above in this manuscript, several recent research efforts and experimental frameworks have begun to address aspects of these challenges.

At the monitoring phase, where the governance layer is encompassed, the contrast with classical systems is stark. Current monitoring tools are largely unaware of quantum-specific properties and offer limited visibility beyond basic job status. There is no standardized way to monitor service-level objectives, reason about cost-performance trade-offs, or enforce governance policies across providers. Pricing, SLAs, and accountability mechanisms remain fragmented and informal, preventing quantum services from being integrated into production-grade service ecosystems.

As can be seen, many components in this architecture are depicted as empty or only partially realized. This is not a weakness of the proposal, but a faithful representation of the current state of the art. The SOQ reference architecture makes explicit where classical service engineering concepts can be reused, where they must be adapted, and where fundamentally new solutions are required. Therefore, by grounding SOQ in a concrete architectural structure rather than an abstract vision, this reference architecture provides a roadmap for future research and engineering.

7.5 SOQ vs. QSOC

The comparison between QSOC and the SOQ, proposed in this manuscript, reveals a fundamental change in how quantum capabilities are conceptualized, integrated, and composed within software architectures. While QSOC was an important first step in adapting classical service-oriented computing principles to support quantum operations, SOQ redefines the architectural foundations by making quantum services native elements of the system design, rather than mere extensions of classical infrastructures.

In a nutshell, and as can be seen in Fig. 6, we believe that these differences between the two paradigms represent a qualitative jump toward treating quantum software as a fundamental element in service-oriented computing. The layered architecture, modular abstractions, orchestration models, and pricing mechanisms introduced by SOQ are key factors for the future of large-scale quantum software engineering.

In QSOC, the architecture is predominantly classical-centric. Quantum components are typically invoked as specialized, back-end services that operate under classical orchestration and control. This model assumes that the quantum functionality is subordinate or complementary to a larger classical workflow. As a result,

Fig. 5. SOQ Reference Architecture.

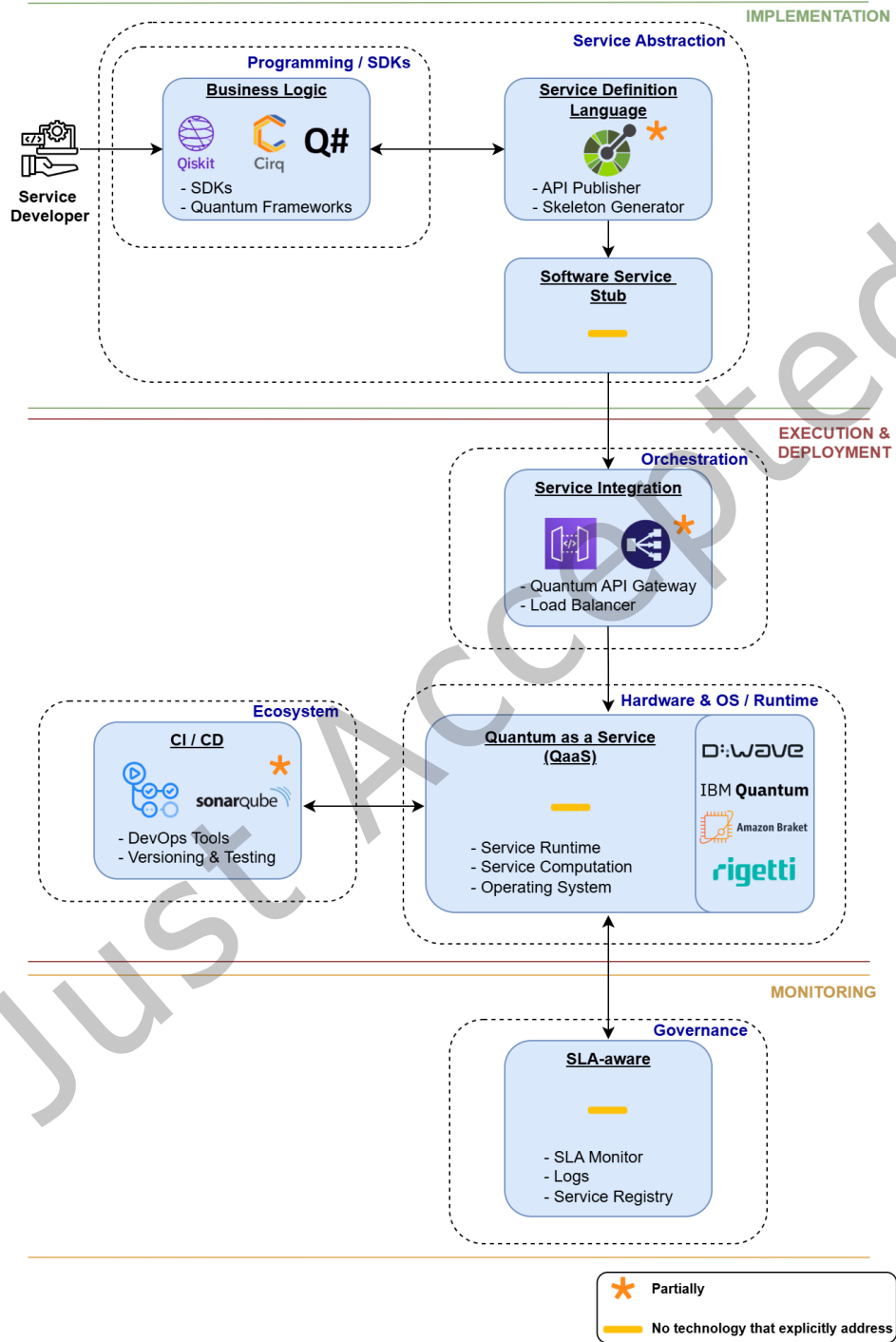


Fig. 6. QSOC vs. SOQ: Key Differences.

QSOC	vs.	SOQ
Extension of classical SOC to include quantum services	Foundation	Native application of SOQ principles to quantum systems
Treated as auxiliary to classical services	Role of Quantum Services	Treated as peer or standalone services
Embedded in classical workflows	Service Composition	Quantum-native or hybrid compositions with independent orchestration
Classical-centric	Architecture	Platform-agnostic, loosely coupled service architecture
Tightly coupled to specific SDKs or cloud providers	Platform Dependency	Abstracted via virtual quantum providers (VQPs)
Classical orchestrators with quantum calls	Orchestration	Hybrid-aware, dynamically scheduled orchestration
Limited by classical control loops	Scalability	Enabled through modular, composable quantum services
Limited abstraction of quality attributes	Service Contracts	SLA-aware interfaces including time, fidelity, and cost
Based on cloud pricing models	Economic Model	Explicit, multi-dimensional pricing for quantum services

service composition, orchestration, and deployment remain bound by the constraints and assumptions of classical software engineering.

In contrast, SOQ advocates a new paradigm where quantum services are autonomous, interoperable, and composable at the same level of abstraction as classical services. Rather than embedding quantum operations within classical applications, SOQ would allow the development of quantum-native service compositions, including pure quantum workflows, hybrid orchestration, and platform-independent service ecosystems. This design would promote dynamic and adaptable quantum software interactions, reusability, and discoverability, which will be essential for scalability and long-term maintainability in quantum software systems.

Another major distinction lies in orchestration and execution control. QSOC relies on classical orchestrators, which may not be optimized to manage quantum-specific concerns. The actual challenge emerges in the interplay

between traditional QoS aspects, typically central to orchestration and binding decisions, and the integration of quantum properties, such as noise sensitivity, probabilistic behavior, execution queue latency, or fidelity guarantees, as part of the QoS of the system. This interplay gives rise to new challenges in ensuring efficient and reliable orchestration in hybrid classical-quantum environments.

This also significantly influences economic models for accessing quantum computing platforms. In QSOC, prices tend to reflect classic cloud-based services, focusing on resource time and performance, although other parameters such as number of qubits, tasks, and/or shots are also included. In contrast, SOQ would allow for the explicit integration of multidimensional pricing models that take into account factors such as fidelity targets, priority execution, or the calibration of the quantum machines themselves, enabling cost-conscious orchestration strategies and more transparent resource governance.

Finally, in terms of platform abstraction, QSOC architectures are typically tied to specific SDKs or quantum back-ends, limiting portability. SOQ introduces layers such as VQPs and platform-agnostic APIs that support seamless integration across heterogeneous quantum hardware and simulators, paving the way for ecosystem-wide interoperability.

8 Conclusion & Future perspectives

This work has introduced and articulated Service-Oriented Quantum (SOQ) as a forward-looking paradigm for structuring quantum software systems using the principles of service orientation. Building upon the limitations identified in the Quantum Service-Oriented Computing (QSOC) model, SOQ emphasizes the importance of treating quantum services as native, first-class components, autonomous, composable, and platform-agnostic. By extending the lessons learned from classical service architectures, SOQ enables modular design, hybrid orchestration, and interoperable deployments, while accounting for the unique characteristics of quantum hardware and software.

To support this vision, we presented a layered SOQ technology stack and a reference architecture for defining quantum services based on the good implementation of a classical service, outlining the necessary components and abstractions ranging from hardware to governance and DevOps. We identified key challenges in areas such as interoperability, hybridity, pricing configuration, circuit modularity, and workforce training, and discussed how addressing these challenges will be critical for the evolution of quantum software engineering. Through case studies and references to recent research, we demonstrated that momentum is already building toward many of these goals, yet much work remains.

Looking ahead, the future of SOQ lies in the co-evolution of quantum and classical infrastructures, and in the creation of a robust ecosystem of services, platforms, and engineering tools. As the industry transitions from NISQ devices toward more scalable and error-resilient quantum hardware, SOQ can serve as the architectural backbone for building sustainable, reusable, and verifiable quantum applications. In parallel, we foresee the emergence of cross-provider orchestration engines, declarative service composition languages for quantum workflows, and AI-driven design and testing tools tailored to the quantum domain.

Ultimately, SOQ provides a practical and extensible framework for integrating quantum computing into the broader landscape of software engineering. It opens new avenues for research in distributed systems, architecture design, economic models, and development methodologies in the quantum era. By grounding quantum software in proven engineering principles while embracing its unique constraints and opportunities, SOQ charts a path toward a scalable, service-driven quantum future.

Acknowledgments

This work has been partially funded by the European Union “Next GenerationEU /PRTR”, by the Ministry of Science, Innovation and Universities (PDC2025-165096-C31). It is also supported by PERSEO project (PID2021-126227NB-C21), and ATHENA project (PID2024-155693NB-C41, PID2024-155693NB-C44) funded by the Spanish Ministry of Science and Innovation and ERDF; and co-financed 85% by the European Union, the European Regional Development Fund and the Regional Government of Extremadura. Managing Authority: Ministry of Finance. Grant File Number: GR24099. Supported by grant PRE2022-102070, financed by MCIN/AEI/10.13039/501100011033, “FEDER/EU”, and FSE+M. Partially supported by “Smart Computer systems Research and Engineering (SCORE)”, funded by the Consejería de Transformación Económica, Industria, Conocimiento y Universidades of Junta de Andalucía as a part of the “Programa de Fortalecimiento de Institutos Universitarios de Investigación de la Universidades Andaluzas, Centros e Infraestructuras para la adquisición del sello ‘Severo Ochoa’ o ‘María de Maeztu’ ”.

References

- [1] Scott Aaronson. 2010. BQP and the polynomial hierarchy. In *Proceedings of the Forty-Second ACM Symposium on Theory of Computing* (Cambridge, Massachusetts, USA) (STOC '10). Association for Computing Machinery, New York, NY, USA, 141–150. doi:10.1145/1806689.1806711
- [2] Aakash Ahmad, Ahmed B Altamimi, and Jamal Aqib. 2024. A reference architecture for quantum computing as a service. *Journal of King Saud University-Computer and Information Sciences* 36, 6 (2024), 1–18. doi:10.1016/j.jksuci.2024.102094
- [3] Shaukat Ali, Johanna Barzen, Andrea Delgado, Hausi A Müller, and Juan Manuel Murillo. 2025. Quantum software engineering. In *National Institute of Informatics (NII), Shonan Village Center, Kanagawa, Japan, Shonan Meeting Report*, Vol. 224. <https://shonan.nii.ac.jp/docs/No.224.pdf>
- [4] Shaukat Ali and Tao Yue. 2023. On the Need of Quantum-Oriented Paradigm. In *Proceedings of the 2nd International Workshop on Quantum Programming for Software Engineering* (San Francisco, CA, USA) (QP4SE 2023). Association for Computing Machinery, New York, NY, USA, 17–20. doi:10.1145/3617570.3617868
- [5] Jaime Alvarado-Valiente, Javier Romero-Álvarez, Ana Díaz, Moisés Rodríguez, Ignacio García-Rodríguez, Enrique Moguel, Jose Garcia-Alonso, and Juan M Murillo. 2023. Quantum services generation and deployment process: a quality-oriented approach. In *International Conference on the Quality of Information and Communications Technology*. Springer, 200–214. doi:10.1007/978-3-031-43703-8_15
- [6] Jaime Alvarado-Valiente, Javier Romero-Álvarez, Jose Garcia-Alonso, and Juan M Murillo. 2022. A guide for quantum web services deployment. In *International Conference on Web Engineering*. Springer, Cham, 493–496. doi:10.1007/978-3-031-09917-5_42
- [7] Jaime Alvarado-Valiente, Javier Romero-Álvarez, Enrique Moguel, and José García-Alonso. 2023. Quantum web services orchestration and management using DevOps techniques. In *International Conference on Web Engineering*. Springer, 389–394. doi:10.1007/978-3-031-34444-2_33
- [8] Jaime Alvarado-Valiente, Javier Romero-Álvarez, Enrique Moguel, Jose García-Alonso, and Juan M Murillo. 2024. Orchestration for quantum services: The power of load balancing across multiple service providers. *Science of Computer Programming* 237 (2024), 103139. doi:10.1016/j.scico.2024.103139
- [9] Jaime Alvarado-Valiente, Javier Romero-Álvarez, Enrique Moguel, José García-Alonso, and Juan M. Murillo. 2024. Technological diversity of quantum computing providers: a comparative study and a proposal for API Gateway integration. *Software Quality Journal* 32, 1 (2024), 53–73. doi:10.1007/s11219-023-09633-5
- [10] Álvaro M Aparicio-Morales, Enrique Moguel, Luis Mariano Bibbo, Alejandro Fernandez, Jose Garcia-Alonso, and Juan M Murillo. 2024. An overview of quantum software engineering in Latin America. *Quantum Information Processing* 23, 11 (2024), 380. doi:10.1007/s11128-024-04586-5
- [11] Kishor Bharti, Alba Cervera-Lierta, Thi Ha Kyaw, Tobias Haug, Sumner Alperin-Lea, Abhinav Anand, Matthias Degroote, Hermann Heimonen, Jakob S. Kottmann, Tim Menke, Wai-Keong Mok, Sukin Sim, Leong-Chuan Kwek, and Alán Aspuru-Guzik. 2022. Noisy intermediate-scale quantum algorithms. *Rev. Mod. Phys.* 94 (2022), 1–69. Issue 1. doi:10.1103/RevModPhys.94.015004
- [12] Tobias Binz, Uwe Breitenbücher, Oliver Kopp, and Frank Leymann. 2013. TOSCA: portable automated deployment and management of cloud applications. In *Advanced Web Services*. Springer, Cham, 527–549. doi:10.1007/978-1-4614-7535-4_22
- [13] Giuseppe Bisicchia, Jaime Alvarado-Valiente, Javier Romero-Álvarez, Jose Garcia-Alonso, Juan M. Murillo, and Antonio Brogi. 2025. Maximizing quantum hardware utilization via multiprogramming circuits and shot-wise distribution. *Information and Software Technology* (2025), 108005. doi:10.1016/j.infsof.2025.108005
- [14] Giuseppe Bisicchia, Alessandro Bocci, and Antonio Brogi. 2025. Quantum Executor: A Unified Interface for Quantum Computing. *arXiv preprint arXiv:2507.07597* 1, 1 (2025), 1–11. doi:10.48550/arXiv.2507.07597

- [15] Giuseppe Bisicchia, Alessandro Bocci, José García-Alonso, Juan M Murillo, and Antonio Brogi. 2024. Cut&Shoot: Cutting & Distributing Quantum Circuits Across Multiple NISQ Computers. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 2. IEEE, Montreal, QC, Canada, 187–192. doi:10.1109/QCE60285.2024.10276
- [16] Giuseppe Bisicchia, Giuseppe Clemente, Jose Garcia-Alonso, Juan Manuel Murillo Rodríguez, Massimo D’Elia, and Antonio Brogi. 2024. Distributing Quantum Computations, Shot-wise. *arXiv preprint arXiv:2411.16530* 1, 1 (2024), 1–22. doi:10.48550/arXiv.2411.16530
- [17] Giuseppe Bisicchia, José García-Alonso, Juan M Murillo, and Antonio Brogi. 2023. Dispatching shots among multiple quantum computers: An architectural proposal. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 2. IEEE, Bellevue, WA, USA, 195–198. doi:10.1109/QCE57702.2023.10210
- [18] Giuseppe Bisicchia, Jose García-Alonso, Juan M Murillo, and Antonio Brogi. 2023. Distributing quantum computations, by shots. In *International Conference on Service-Oriented Computing*. Springer, Cham, 363–377. doi:10.1007/978-3-031-48421-6_25
- [19] Giuseppe Bisicchia, Jose García-Alonso, Juan M. Murillo, and Antonio Brogi. 2024. From Quantum Software Handcrafting to Quantum Software Engineering. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion (SANER-C)*. IEEE, Rovaniemi, Finland, 149–150. doi:10.1109/SANER-C62648.2024.00026
- [20] Gerardo Canfora, Massimiliano Di Penta, Raffaele Esposito, and Maria Luisa Villani. 2008. A framework for QoS-aware binding and re-binding of composite web services. *Journal of Systems and Software* 81, 10 (2008), 1754–1769.
- [21] Isaac L. Chuang, Neil Gershenfeld, and Mark Kubinec. 1998. Experimental Implementation of Fast Quantum Searching. *Phys. Rev. Lett.* 80 (1998), 3408–3411. Issue 15. doi:10.1103/PhysRevLett.80.3408
- [22] John Clark and Susan Stepney. 2002. *Proposed "Grand Challenge for Computing Research" Quantum Software Engineering*. Technical Report. University of York. <https://www.cs.york.ac.uk/nature/gc7/journeys.pdf>
- [23] Sandeep Suresh Cranganore, Vincenzo De Maio, Ivona Brandic, and Ewa Deelman. 2024. Paving the way to hybrid quantum-classical scientific workflows. *Future Generation Computer Systems* 158 (2024), 346–366. doi:10.1016/j.future.2024.04.030
- [24] Andrew Cross, Ali Javadi-Abhari, Thomas Alexander, Niel De Beaudrap, Lev S Bishop, Steven Heidel, Colm A Ryan, Prasahnt Sivarajah, John Smolin, Jay M Gambetta, et al. 2022. OpenQASM 3: A broader and deeper quantum assembly language. *ACM Transactions on Quantum Computing* 3, 3 (2022), 1–50. doi:10.1145/3505636
- [25] Andrea Delgado and Prasanna Date. 2025. Defining quantum-ready primitives for hybrid HPC-QC supercomputing: a case study in Hamiltonian simulation. *Frontiers in Computer Science* 7 (2025), 1528985. doi:10.3389/fcomp.2025.1528985
- [26] D. Deutsch. 1985. Quantum theory, the Church-Turing principle and the universal quantum computer. *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences* 400 (1985), 97–117. Issue 1818. doi:10.1098/RSPA.1985.0070
- [27] Ana Díaz, Jaime Alvarado-Valiente, Javier Romero-Álvarez, Enrique Moguel, Jose Garcia-Alonso, Moisés Rodríguez, Ignacio García-Rodríguez, and Juan M Murillo. 2025. Service engineering for quantum computing: Ensuring high-quality quantum services. *Information and Software Technology* 179 (2025), 1–13. doi:10.1016/j.infsof.2024.107643
- [28] Suguru Endo, Zhenyu Cai, Simon C Benjamin, and Xiao Yuan. 2021. Hybrid quantum-classical algorithms and quantum error mitigation. *Journal of the Physical Society of Japan* 90, 3 (2021), 032001. doi:10.7566/JPSJ.90.032001
- [29] Michael Falkenthal, Christoph Krieger, Felix Paul, Sebastian Wagner, and Michael Wurster. 2024. Planqk—platform and ecosystem for quantum applications. *KI-Künstliche Intelligenz* 38, 4 (2024), 371–377. doi:10.1007/s13218-024-00865-6
- [30] Ismael Faro, Iskandar Sitdikov, David Garcia Valiñas, Francisco Jose Martin Fernandez, Christopher Codella, and Jennifer Glick. 2023. Middleware for Quantum: An orchestration of hybrid quantum-classical systems. In *2023 IEEE International Conference on Quantum Software (QSW)*. IEEE, Chicago, IL, USA, 1–8. doi:10.1109/QSW59989.2023.00011
- [31] Richard P. Feynman. 1982. Simulating physics with computers. *International Journal of Theoretical Physics* 21, 6 (1982), 467–488. doi:10.1007/BF02650179
- [32] Rafael Fresno-Aranda, Pablo Fernández, Amador Durán, and Antonio Ruiz-Cortés. 2022. Semi-automated capacity analysis of limitation-aware microservices architectures. In *International Conference on the Economics of Grids, Clouds, Systems, and Services*. Springer, 75–88. doi:10.1007/978-3-031-29315-3_7
- [33] Antonio Gamez-Diaz, Pablo Fernandez, and Antonio Ruiz-Cortes. 2018. SLA-Driven Governance for RESTful Systems. In *Service-Oriented Computing, ICSOC 2017 Workshops*, Lars Braubach, Juan M. Murillo, Nima Kavian, Manuel Lama, Loli Burgueño, Naouel Moha, and Marc Oriol (Eds.). Springer, Cham, 352–356. doi:10.1007/978-3-319-91764-1_30
- [34] Jose Garcia-Alonso, Javier Rojo, David Valencia, Enrique Moguel, Javier Berrocal, and Juan Manuel Murillo. 2021. Quantum software as a service through a quantum API gateway. *IEEE Internet Computing* 26, 1 (2021), 34–41. doi:10.1109/MIC.2021.3132688
- [35] Alejandro García-Fernández, José Antonio Parejo, Francisco Javier Cavero, and Antonio Ruiz-Cortés. 2024. Racing the Market: An Industry Support Analysis for Pricing-Driven DevOps in SaaS. In *International Conference on Service-Oriented Computing*. Springer, Cham, 260–275. doi:10.1007/978-981-96-0808-9_19
- [36] Alejandro García-Fernández, José Antonio Parejo, Pablo Trinidad, and Antonio Ruiz-Cortés. 2025. Automated Analysis of Pricings in SaaS-Based Information Systems. In *Advanced Information Systems Engineering*, John Krogstie, Stefanie Rinderle-Ma, Gerti Kappel, and Henderik A. Proper (Eds.). Springer, Cham, 223–239.

- [37] Emmanouil Giortamis, Francisco Romão, Nathaniel Törnøw, and Pramod Bhatotia. 2025. {QOS}: Quantum Operating System. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. 429–447. doi:10.48550/arXiv.2406.19120
- [38] Tudor Giurgica-Tiron, Yousef Hindy, Ryan LaRose, Andrea Mari, and William J Zeng. 2020. Digital zero noise extrapolation for quantum error mitigation. In *2020 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, 306–316. doi:10.1109/QCE49297.2020.00045
- [39] Michele Grossi, Luca Crippa, Antonello Aita, Giacomo Bartoli, Vito Sammarco, Eleonora Picca, Najla Said, Filippo Tramonto, and Federico Mattei. 2021. A serverless cloud integration for quantum computing. *arXiv preprint arXiv:2107.02007* 1, 1 (2021), 1–8. doi:10.48550/arXiv.2107.02007
- [40] Lov K. Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (Philadelphia, Pennsylvania, USA) (STOC '96)*. Association for Computing Machinery, New York, NY, USA, 212–219. doi:10.1145/237814.237866
- [41] Xiaoyu Guo, Shinobu Saito, and Jianjun Zhao. 2025. QuanUML: Towards a modeling language for model-driven quantum software development. In *2025 IEEE 49th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 1344–1349. doi:10.1109/COMPSAC65507.2025.00168
- [42] Majid Haghparast, Enrique Moguel, Jose Garcia-Alonso, Tommi Mikkonen, and Juan Manuel Murillo. 2024. Innovative Approaches to Teaching Quantum Computer Programming and Quantum Software Engineering. *Proceedings - IEEE Quantum Week 2024, QCE 2024 2* (2024), 251–255. doi:10.1109/QCE60285.2024.10287
- [43] Sovanmonynuth Heng, Dongmin Kim, Taekyung Kim, and Youngsun Han. 2022. How to Solve Combinatorial Optimization Problems Using Real Quantum Machines: A Recent Survey. *IEEE Access* 10 (2022), 120106–120121. doi:10.1109/ACCESS.2022.3218908
- [44] Hirofumi Irie, Haozhao Liang, Takumi Doi, Shinya Gongyo, and Tetsuo Hatsuda. 2021. Hybrid quantum annealing via molecular dynamics. *Scientific reports* 11, 1 (2021), 1–9. doi:10.1038/s41598-021-87676-z
- [45] Chandrashekar Jatoth and G. R. Gangadharan. 2015. QoS-Aware Web Service Composition Using Quantum Inspired Particle Swarm Optimization. In *Intelligent Decision Technologies*, Rui Neves-Silva, Lakshmi C. Jain, and Robert J. Howlett (Eds.). Springer, Cham, 255–265. doi:10.1007/978-3-319-19857-6_23
- [46] Martin J Klein. 1961. Max Planck and the beginnings of the quantum theory. *Archive for History of Exact Sciences* 1, 5 (1961), 459–479. doi:10.1007/BF00327765
- [47] Emanuel Knill. 2010. Quantum computing. *Nature* 463 (2010), 441–443. doi:10.1038/463441a
- [48] Indika Kumara, Willem-Jan Van Den Heuvel, and Damian A Tamburri. 2021. QSOC: Quantum service-oriented computing. In *Symposium and Summer School on Service-Oriented Computing*. Springer, Cham, 52–63. doi:10.1007/978-3-030-87568-8_3
- [49] Thaddeus D Ladd, Fedor Jelezko, Raymond Laflamme, Yasunobu Nakamura, Christopher Monroe, and Jeremy Lloyd O'Brien. 2010. Quantum computers. *nature* 464, 7285 (2010), 45–53. doi:10.1038/nature08812
- [50] Neilson Carlos Leite Ramalho, Higor Amario de Souza, and Marcos Lordello Chaim. 2025. Testing and Debugging Quantum Programs: The Road to 2030. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (2025), 1–46. doi:10.1145/3715106
- [51] Frank Leymann. 2019. Towards a pattern language for quantum algorithms. In *Quantum Technology and Optimization Problems: First International Workshop, QTOP 2019, Munich, Germany, March 18, 2019, Proceedings 1*. Springer, Cham, 218–230. doi:10.1007/978-3-030-14082-3_19
- [52] Frank Leymann and Johanna Barzen. 2020. The bitter truth about gate-based quantum algorithms in the NISQ era. *Quantum Science and Technology* 5, 4 (2020), 1–29. doi:10.1088/2058-9565/abae7d
- [53] Junjie Luo and Jianjun Zhao. 2025. Formalization of quantum intermediate representations for code safety. *Journal of Systems and Software* 219 (2025), 1–12. doi:10.1016/j.jss.2024.112236
- [54] Alexander J McCaskey, Dmitry I Lyakh, Eugene F Dumitrescu, Sarah S Powers, and Travis S Humble. 2020. XACC: a system-level software infrastructure for heterogeneous quantum-classical computing. *Quantum Science and Technology* 5, 2 (2020), 1–24. doi:10.1088/2058-9565/ab6bf6
- [55] Jarrod R McClean, Jonathan Romero, Ryan Babbush, and Alán Aspuru-Guzik. 2016. The theory of variational hybrid quantum-classical algorithms. *New Journal of Physics* 18, 2 (2016), 1–23. doi:10.1088/1367-2630/18/2/023023
- [56] Enrique Moguel, Javier Rojo, David Valencia, Javier Berrocal, Jose Garcia-Alonso, and Juan M Murillo. 2022. Quantum service-oriented computing: current landscape and challenges. *Software Quality Journal* 30, 4 (2022), 983–1002. doi:10.1007/S11219-022-09589-Y
- [57] Asmar Muqeet, Tao Yue, Shaikat Ali, and Paolo Arcaini. 2024. Mitigating Noise in Quantum Software Testing Using Machine Learning. *IEEE Transactions on Software Engineering* 50, 11 (2024), 2947–2961. doi:10.1109/TSE.2024.3462974
- [58] Juan M Murillo, Jose Garcia-Alonso, Enrique Moguel, Johanna Barzen, Frank Leymann, Shaikat Ali, Tao Yue, Paolo Arcaini, Ricardo Pérez-Castillo, Ignacio García Rodríguez de Guzmán, Mario Piattini, Antonio Ruiz-Cortes, Antonio Brogi, Jianjun Zhao, Andriy Miransky, and Manuel Wimmer. 2025. Quantum Software Engineering: Roadmap and Challenges Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–48. doi:10.1145/3712002
- [59] Hoa T Nguyen, Prabhakar Krishnan, Dilip Krishnaswamy, Muhammad Usman, and Rajkumar Buyya. 2024. Quantum Cloud Computing: A Review, Open Problems, and Future Directions. *arXiv preprint arXiv:2404.11420* (2024). doi:10.48550/arXiv.2404.11420

- [60] Hoa T Nguyen, Muhammad Usman, and Rajkumar Buyya. 2024. Qfaas: A serverless function-as-a-service framework for quantum computing. *Future Generation Computer Systems* 154 (2024), 281–300. doi:10.1016/j.future.2024.01.018
- [61] Michael A Nielsen and Isaac L Chuang. 2010. *Quantum computation and quantum information*. Cambridge university press, Cambridge. doi:10.1017/CBO9780511976667
- [62] Roman Orus, Samuel Mugel, and Enrique Lizaso. 2019. Quantum computing for finance: overview and prospects. *Reviews in Physics* 4 (2019), 1–12. doi:10.1016/j.revip.2019.100028
- [63] Lee J O’Riordan, Myles Doyle, Fabio Baruffa, and Venkatesh Kannan. 2020. A hybrid classical-quantum workflow for natural language processing. *Machine Learning: Science and Technology* 2, 1 (2020), 1–25. doi:10.1088/2632-2153/abbd2e
- [64] José Antonio Parejo and Antonio Ruiz-Cortés. 2025. The Advantage Boundary: A Formal Basis for Integrating Quantum and Classical Services in Hybrid SaaS. In *Service-Oriented Computing, ICSOC 2025 Workshops*. Springer. In Press.
- [65] Mritunjay Shall Peelam, Anjaney Asreet Rout, and Vinay Chamola. 2024. Quantum computing applications for Internet of Things. *IET Quantum Communication* 5, 2 (2024), 103–112. doi:10.1049/qtc2.12079
- [66] Ricardo Pérez-Castillo and Mario Piattini. 2022. Design of classical-quantum systems with UML. *Computing* 104, 11 (2022), 2375–2403. doi:10.1007/s00607-022-01091-4
- [67] Aage Petersen. 1963. The philosophy of niels bohr. *Bulletin of the atomic scientists* 19, 7 (1963), 8–14. doi:10.1080/00963402.1963.11454520
- [68] Mario Piattini, Guido Petersen, Ricardo Pérez-Castillo, Jose Luis Hevia, Manuel A Serrano, Guillermo Hernández, Ignacio García Rodríguez De Guzmán, Claudio Andrés Paradela, Macario Polo, Ezequiel Murina, et al. 2020. The Talavera Manifesto for quantum software engineering and programming.. In *QANSWER*. CEUR, CEUR-WS.org, 1–5. <https://ceur-ws.org/Vol-2561/paper0.pdf>
- [69] John Preskill. 2018. Quantum Computing in the NISQ era and beyond. *Quantum* 2 (2018), 1–20. doi:10.22331/q-2018-08-06-79
- [70] Nils Quetschlich, Lukas Burgholzer, and Robert Wille. 2023. Predicting Good Quantum Circuit Compilation Options. In *2023 IEEE International Conference on Quantum Software (QSW)*. IEEE, Chicago, IL, USA, 43–53. doi:10.1109/QSW59989.2023.00015
- [71] Antonio Quiña Mera, Pablo Fernandez, José María García, and Antonio Ruiz-Cortés. 2023. GraphQL: A Systematic Mapping Study. *ACM Comput. Surv.* 55, 10 (2023), 1–35. doi:10.1145/3561818
- [72] Gokul Subramanian Ravi, Kaitlin N. Smith, Prakash Murali, and Frederic T. Chong. 2021. Adaptive job and resource management for the growing quantum cloud. In *2021 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, Broomfield, CO, USA, 301–312. doi:10.1109/QCE52317.2021.00047
- [73] Max Riedel, Matyas Kovacs, Peter Zoller, Jürgen Mlynek, and Tommaso Calarco. 2019. Europe’s quantum flagship initiative. *Quantum Science and Technology* 4, 2 (2019), 1–7. doi:10.1088/2058-9565/ab042d
- [74] Javier Romero-Álvarez, Jaime Alvarado-Valiente, Jorge Casco-Seco, Enrique Moguel, Jose Garcia-Alonso, and Juan M Murillo. 2024. A noise validation for quantum circuit scheduling through a service-oriented architecture. *International Journal of Software Engineering and Knowledge Engineering* 34, 09 (2024), 1371–1386. doi:10.1142/s0218194024410018
- [75] Javier Romero-Álvarez, Jaime Alvarado-Valiente, Jose Garcia-Alonso, Enrique Moguel, and Juan M Murillo. 2021. A graph-based healthcare system for quantum simulation of medication administration in the aging people. In *International Workshop on Gerontechnology*. Springer, Cham, 34–41. doi:10.1007/978-3-030-97524-1_4
- [76] Javier Romero-Álvarez, Jaime Alvarado-Valiente, Enrique Moguel, José García-Alonso, and Juan M Murillo. 2022. Using open API for the development of hybrid classical-quantum services. In *International conference on service-oriented computing*. Springer, 364–368. doi:10.1007/978-3-031-26507-5_34
- [77] Javier Romero-Álvarez, Jaime Alvarado-Valiente, Enrique Moguel, Jose Garcia-Alonso, and Juan M Murillo. 2024. Enabling continuous deployment techniques for quantum services. *Software: Practice and Experience* 54, 8 (2024), 1491–1515. doi:10.1002/spe.3326
- [78] Javier Romero-Álvarez, Jaime Alvarado-Valiente, Enrique Moguel, José Garcia-Alonso, and Juan M Murillo. 2024. *Quantum Service-oriented Computing: A Proposal for Quantum Software as a Service*. River Publishers, London. doi:10.1201/9788770046336
- [79] Javier Romero-Álvarez, Jaime Alvarado-Valiente, Enrique Moguel, Carlos Canal, Jose Garcia-Alonso, and Juan M. Murillo. 2023. Leveraging API Specifications for Scaffolding Quantum Applications. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 02. IEEE, Bellevue, WA, USA, 187–190. doi:10.1109/QCE57702.2023.10208
- [80] Antonio Ruiz-Cortés and José Antonio Parejo. 2025. An Initial Exploration of Pricing-driven Governance for Hybrid Quantum-Classical SaaS. In *XXIX Jornadas de Ingeniería del Software y Bases de Datos (JISBD) - Track on Quantum Computing and Quantum Software Engineering (QuantumX)*. SISTEDES, Córdoba, Spain, 1–13. <https://hdl.handle.net/11705/JISBD/2025/91>
- [81] Marie Salm, Johanna Barzen, Uwe Breitenbücher, Frank Leymann, Benjamin Weder, and Karoline Wild. 2020. The NISQ analyzer: automating the selection of quantum computers for quantum algorithms. In *Symposium and summer school on Service-Oriented Computing*. Springer, Cham, 66–85. doi:10.1007/978-3-030-64846-6_5
- [82] Marie Salm, Johanna Barzen, Frank Leymann, and Philipp Wundrack. 2022. Optimizing the prioritization of compiled quantum circuits by machine learning approaches. In *Symposium and Summer School on Service-Oriented Computing*. Springer, Cham, 161–181. doi:10.1007/978-3-031-18304-1_9
- [83] Javier Sanchez-Rivero, Daniel Talaván, Jose Garcia-Alonso, Antonio Ruiz-Cortés, and Juan Manuel Murillo. 2025. Automatic generation of efficient oracles: The less-than case. *Journal of Systems and Software* 219 (2025), 112203. doi:10.1016/j.jss.2024.112203

- [84] Nishant Saurabh, Shantenu Jha, and Andre Luckow. 2023. A conceptual architecture for a quantum-hpc middleware. In *2023 IEEE international conference on Quantum Software (QSW)*. IEEE, 116–127. doi:10.1109/QSW59989.2023.00023
- [85] Manuel A Serrano, José A Cruz-Lemus, Ricardo Perez-Castillo, and Mario Piattini. 2022. Quantum software components and platforms: Overview and quality assessment. *Comput. Surveys* 55, 8 (2022), 1–31. doi:10.1145/3548679
- [86] Peter W. Shor. 1997. Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer. *SIAM J. Comput.* 26, 5 (1997), 1484–1509. doi:10.1137/S0097539795293172
- [87] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. 2020. t|ket>: a retargetable compiler for NISQ devices. *Quantum Science and Technology* 6, 1 (2020), 1–28. doi:10.1088/2058-9565/ab8e92
- [88] Sandeep Kumar Sood and Monika Agrewal. 2024. Quantum machine learning for computational methods in engineering: a systematic review. *Archives of Computational Methods in Engineering* 31, 3 (2024), 1555–1577. doi:10.1007/s11831-023-10027-w
- [89] Vlad Stirbu, Otso Kinanen, Majid Haghparsat, and Tommi Mikkonen. 2024. Qubernetes: Towards a unified cloud-native execution platform for hybrid classic-quantum computing. *Information and Software Technology* 175 (2024), 1–11. doi:10.1016/j.infsof.2024.107529
- [90] David Valencia, Jose Garcia-Alonso, Javier Rojo, Enrique Moguel, Javier Berrocal, and Juan Manuel Murillo. 2021. Hybrid classical-quantum software services systems: Exploration of the rough edges. In *International Conference on the Quality of Information and Communications Technology*. Springer, 225–238. doi:10.1007/978-3-030-85347-1_17
- [91] Benjamin Weder, Johanna Barzen, Frank Leymann, and Daniel Vietz. 2022. Quantum software development lifecycle. In *Quantum Software Engineering*. Springer, 61–83. doi:10.1007/978-3-031-05324-5_4
- [92] Benjamin Weder, Uwe Breitenbücher, Frank Leymann, and Karoline Wild. 2020. Integrating quantum computing into workflow modeling and execution. In *2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC)*. IEEE, Leicester, UK, 279–291. doi:10.1109/UCC48980.2020.00046
- [93] Yi Wei and M. Brian Blake. 2010. Service-Oriented Computing and Cloud Computing: Challenges and Opportunities. *IEEE Internet Computing* 14, 6 (2010), 72–75. doi:10.1109/MIC.2010.147
- [94] Karoline Wild, Uwe Breitenbücher, Lukas Harzenetter, Frank Leymann, Daniel Vietz, and Michael Zimmermann. 2020. TOSCA4QC: two modeling styles for TOSCA to automate the deployment and orchestration of quantum applications. In *2020 IEEE 24th International Enterprise Distributed Object Computing Conference (EDOC)*. IEEE, Eindhoven, Netherlands, 125–134. doi:10.1109/EDOC49727.2020.00024
- [95] Nouredine Zettili and Ismail Zahed. 2003. Quantum Mechanics: Concepts and Applications. *American Journal of Physics* 71, 1 (2003), 93–93. doi:10.1119/1.1522702
- [96] Yun Zhang, Ram Krishnan, and Ravi Sandhu. 2014. Secure Information and Resource Sharing in Cloud Infrastructure as a Service. In *Proceedings of the 2014 ACM Workshop on Information Sharing & Collaborative Security* (Scottsdale, Arizona, USA) (WISCS '14). Association for Computing Machinery, New York, NY, USA, 81–90. doi:10.1145/2663876.2663884
- [97] Jianjun Zhao. 2020. Quantum software engineering: Landscapes and horizons. *arXiv preprint arXiv:2007.07047* 1, 1 (2020), 1–34. doi:10.48550/arXiv.2007.07047
- [98] Álvaro M. Aparicio-Morales, Enrique Moguel, José García-Alonso, Alejandro Fernandez, Luis Mariano Bibbo, and Juan M. Murillo. 2024. Oferta y demanda en la formación de personal para la ingeniería de software cuántico. *Memoria Investigaciones en Ingeniería* 1, 1 (2024), 248–256. Issue 27. doi:10.36561/ING.27.16